

Soham Kharabe

C_C2_23

AI Lab

Practical 4

11/08/2026

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
```

```
In [2]: df = pd.read_csv("winequality-red.csv")
```

```
In [3]: df.head()
```

```
Out[3]:
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	a
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	



```
In [4]: df.tail()
```

Out[4]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphate
1594	6.2	0.600	0.08	2.0	0.090	32.0	44.0	0.99490	3.45	0.51
1595	5.9	0.550	0.10	2.2	0.062	39.0	51.0	0.99512	3.52	0.70
1596	6.3	0.510	0.13	2.3	0.076	29.0	40.0	0.99574	3.42	0.71
1597	5.9	0.645	0.12	2.0	0.075	32.0	44.0	0.99547	3.57	0.71
1598	6.0	0.310	0.47	3.6	0.067	18.0	42.0	0.99549	3.39	0.60



In [5]: `df.shape`

Out[5]: (1599, 12)

In [6]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1599 entries, 0 to 1598
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   fixed acidity          1599 non-null   float64
1   volatile acidity       1599 non-null   float64
2   citric acid            1599 non-null   float64
3   residual sugar         1599 non-null   float64
4   chlorides              1599 non-null   float64
5   free sulfur dioxide    1599 non-null   float64
6   total sulfur dioxide   1599 non-null   float64
7   density                1599 non-null   float64
8   pH                    1599 non-null   float64
9   sulphates              1599 non-null   float64
10  alcohol                1599 non-null   float64
11  quality                1599 non-null   int64   
dtypes: float64(11), int64(1)
memory usage: 150.0 KB
```

In [7]: `df.describe()`

Out[7]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide
count	1599.000000	1599.000000	1599.000000	1599.000000	1599.000000	1599.000000	1599
mean	8.319637	0.527821	0.270976	2.538806	0.087467	15.874922	46
std	1.741096	0.179060	0.194801	1.409928	0.047065	10.460157	32
min	4.600000	0.120000	0.000000	0.900000	0.012000	1.000000	6
25%	7.100000	0.390000	0.090000	1.900000	0.070000	7.000000	22
50%	7.900000	0.520000	0.260000	2.200000	0.079000	14.000000	38
75%	9.200000	0.640000	0.420000	2.600000	0.090000	21.000000	62
max	15.900000	1.580000	1.000000	15.500000	0.611000	72.000000	289



In [8]: `df.isnull().sum()`

Out[8]:

```
fixed acidity      0
volatile acidity  0
citric acid        0
residual sugar     0
chlorides          0
free sulfur dioxide 0
total sulfur dioxide 0
density           0
pH               0
sulphates         0
alcohol           0
quality           0
dtype: int64
```

In [9]: `df.duplicated().sum()`

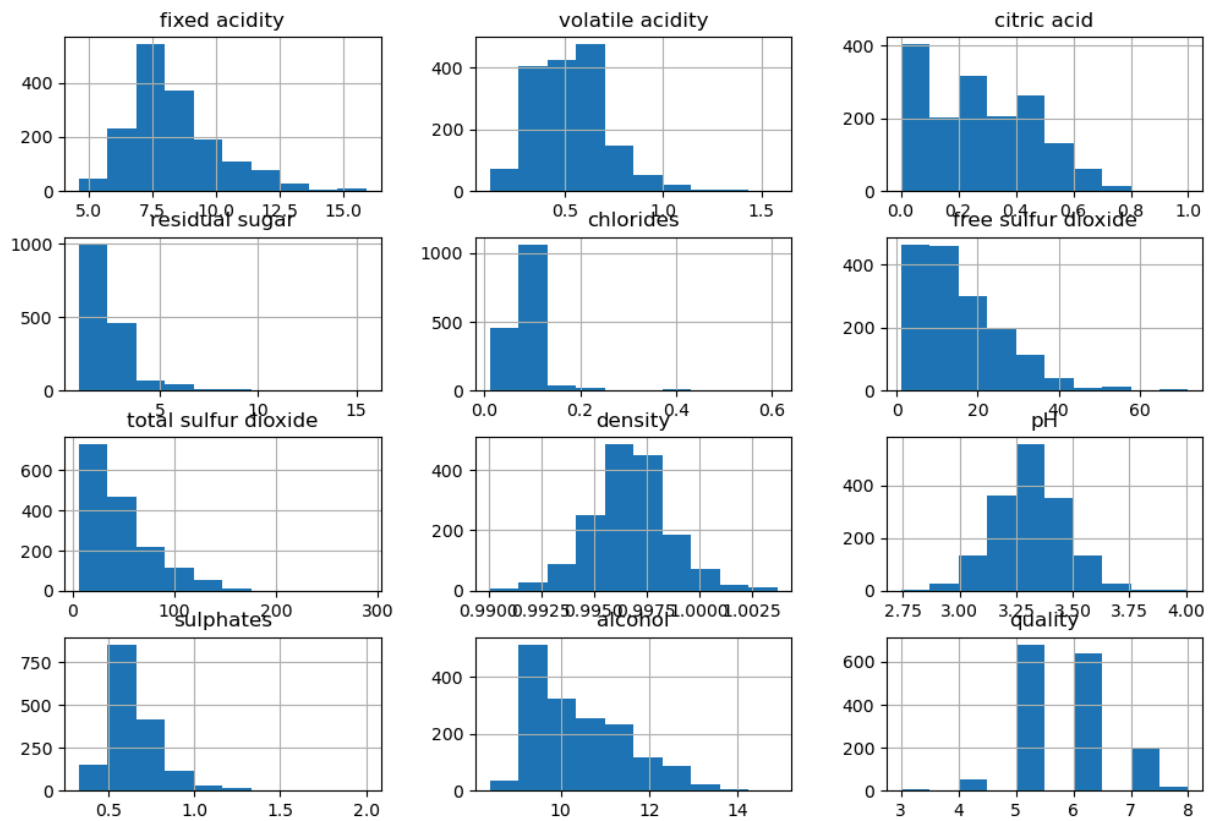
Out[9]: `np.int64(240)`

In [10]: `df.duplicated().sum()`

Out[10]: `np.int64(240)`

In [11]: `df.hist(figsize=(12,8))`
`plt.show`

Out[11]: `<function matplotlib.pyplot.show(close=None, block=None)>`



```
In [12]: df.head()
```

```
Out[12]:
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	



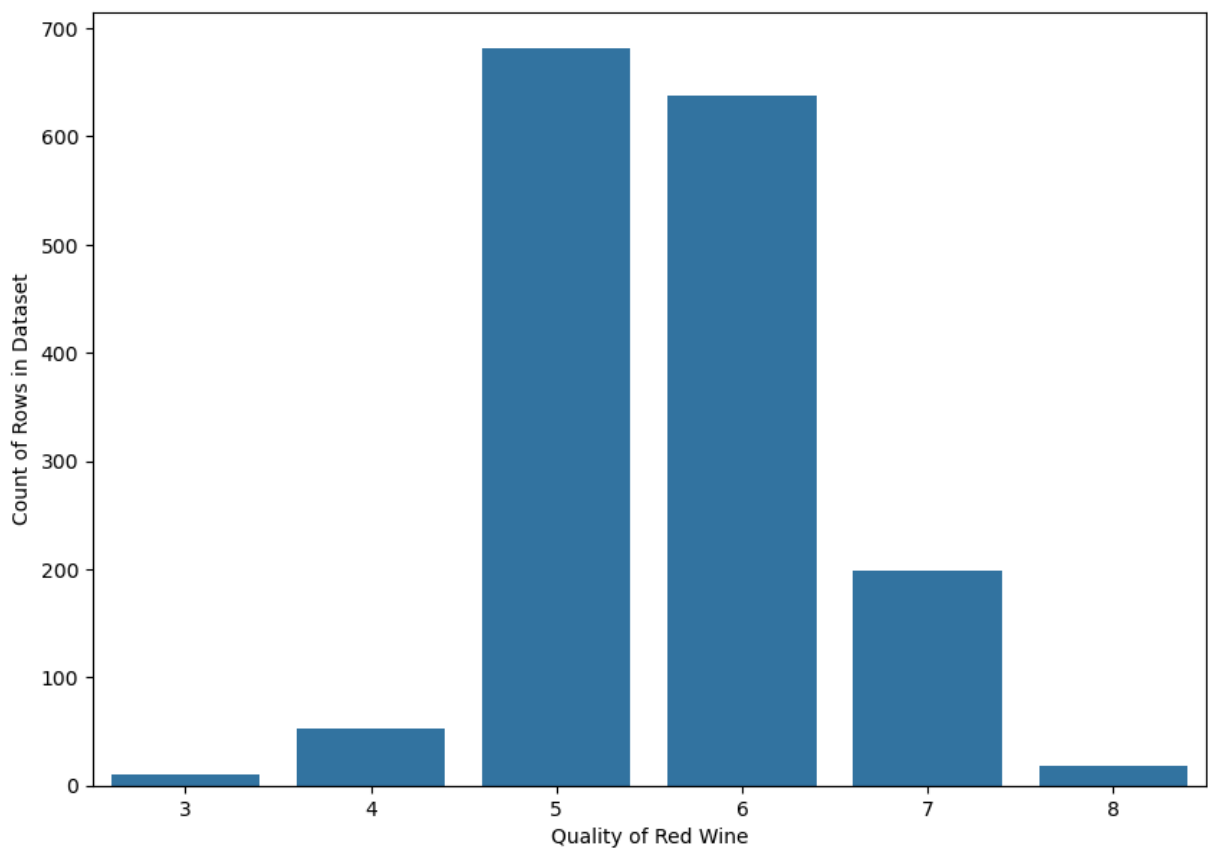
```
In [13]: df.duplicated().sum
```

```
Out[13]: <bound method Series.sum of 0      False
1      False
2      False
3      False
4      True
...
1594   False
1595   False
1596    True
1597   False
1598   False
Length: 1599, dtype: bool>
```

Univariant Analysis

CountPlot

```
In [14]: plt.figure(figsize=(10,7))
sns.countplot(x='quality',data=df)
plt.xlabel('Quality of Red Wine')
plt.ylabel('Count of Rows in Dataset')
plt.show()
```



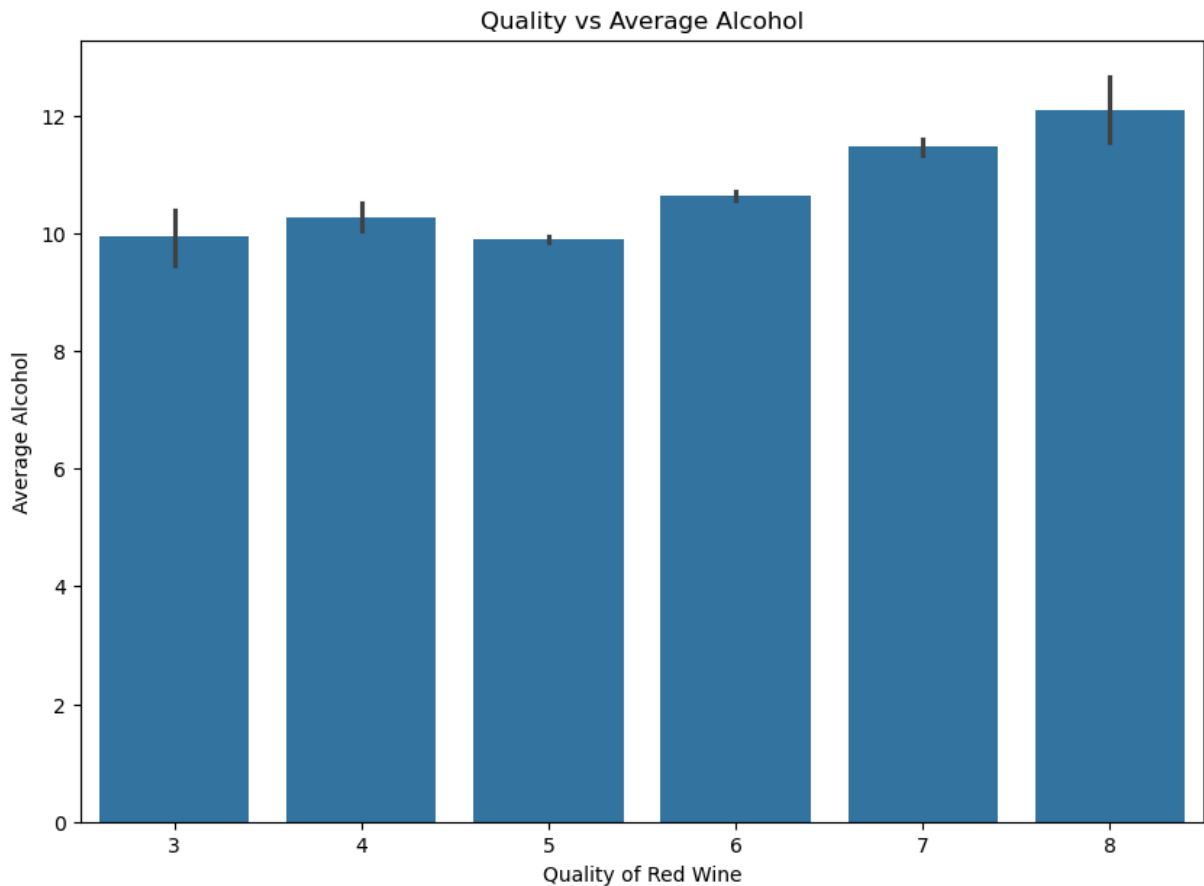
Bar Plot

```
In [15]: plt.figure(figsize=(10,7))

sns.barplot(x="quality", y="alcohol", data=df)

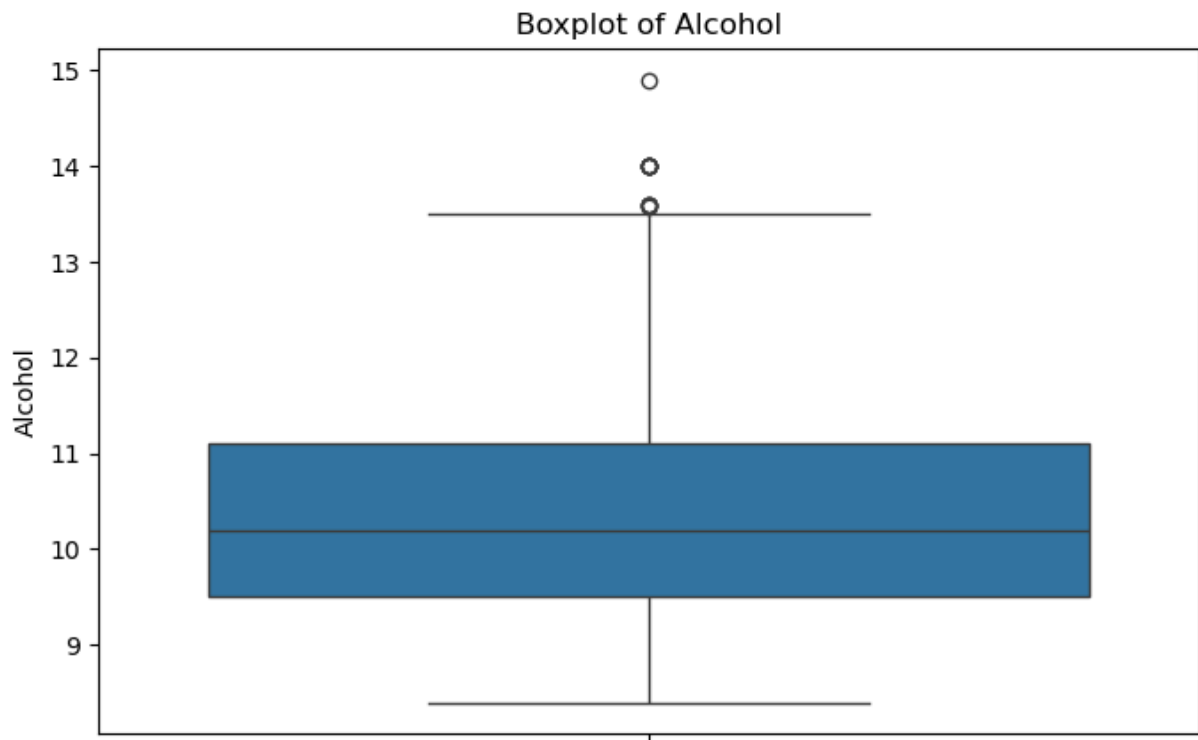
plt.xlabel("Quality of Red Wine")
plt.ylabel("Average Alcohol")
plt.title("Quality vs Average Alcohol")

plt.show()
```

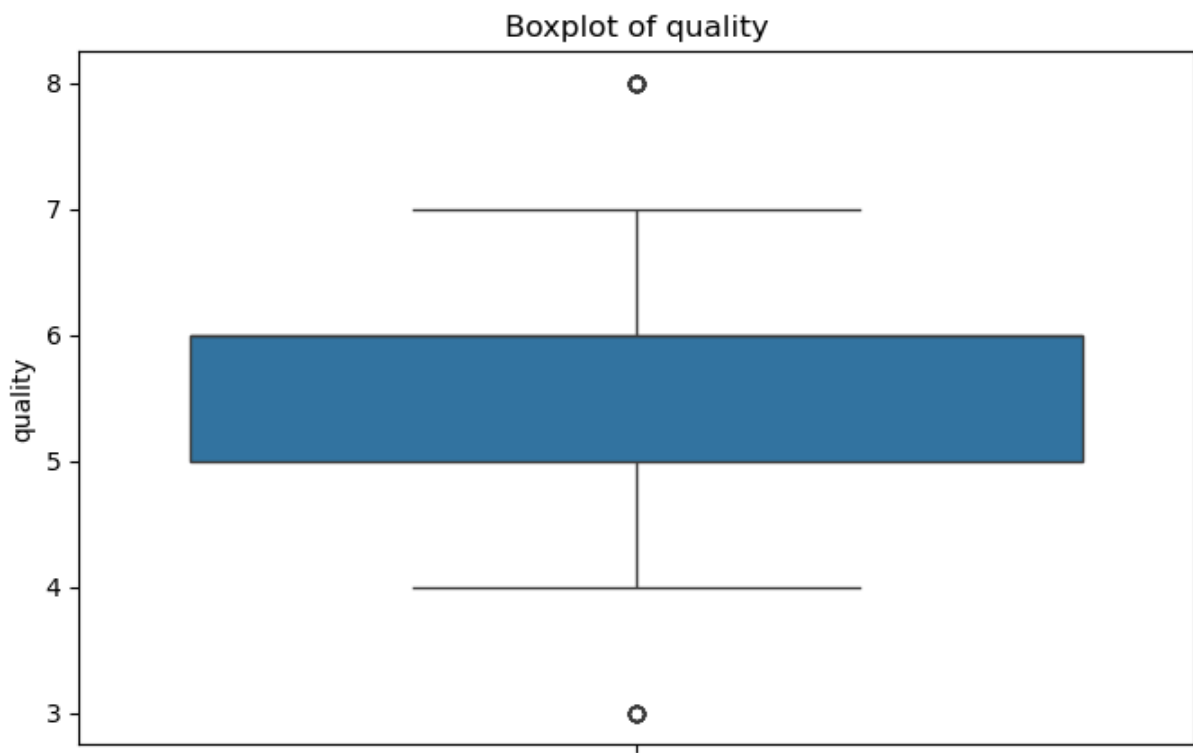


Box Plot

```
In [16]: plt.figure(figsize=(8, 5))
sns.boxplot(y=df["alcohol"])
plt.title("Boxplot of Alcohol")
plt.ylabel("Alcohol")
plt.show()
```

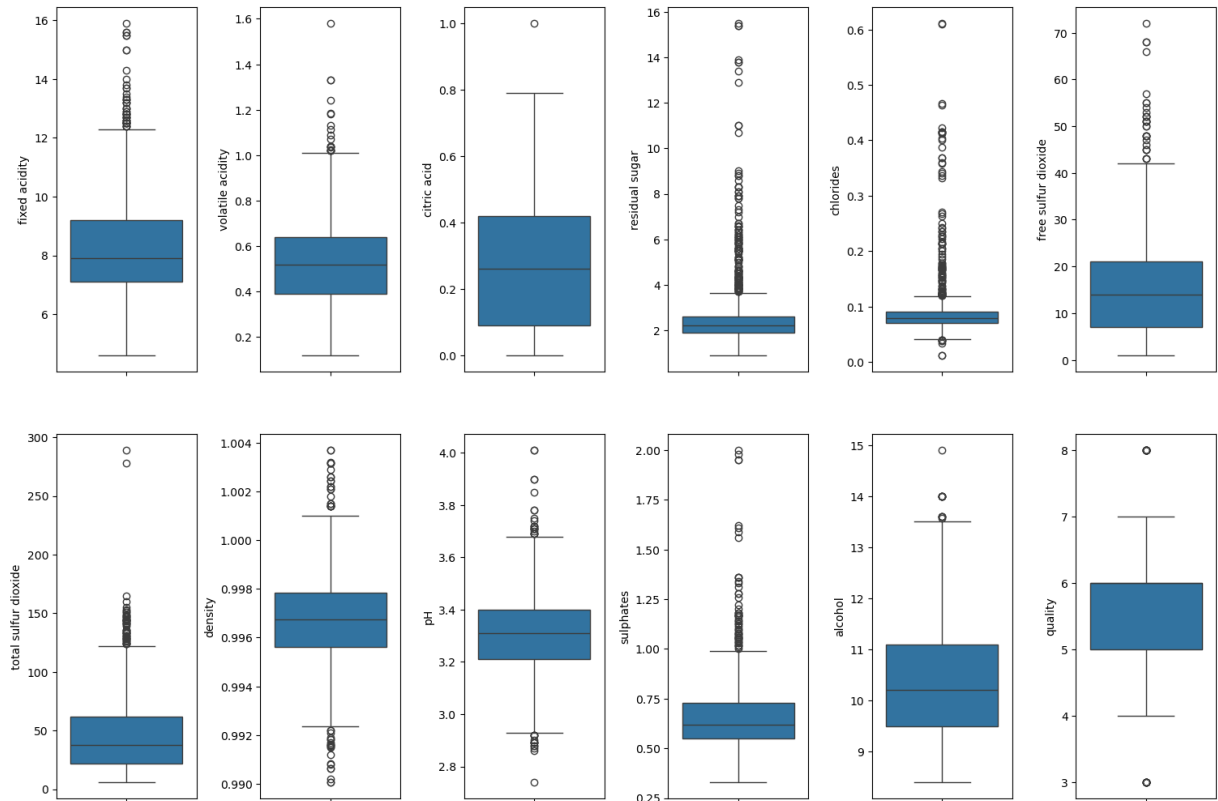


```
In [17]: plt.figure(figsize=(8, 5))  
sns.boxplot(y=df["quality"])  
plt.title("Boxplot of quality")  
plt.ylabel("quality")  
plt.show()
```



Univariate Analysis (Box Plot)

```
In [18]: fig, ax=plt.subplots(ncols=6, nrows=2, figsize=(15,10))
index=0
ax=ax.flatten()
for col,value in df.items():
    sns.boxplot(y=col,data=df,ax=ax[index])
    index +=1
plt.tight_layout(pad=0.5,w_pad=0.7,h_pad=5.0)
plt.show()
```



Univariate Analysis (Dis Plot)

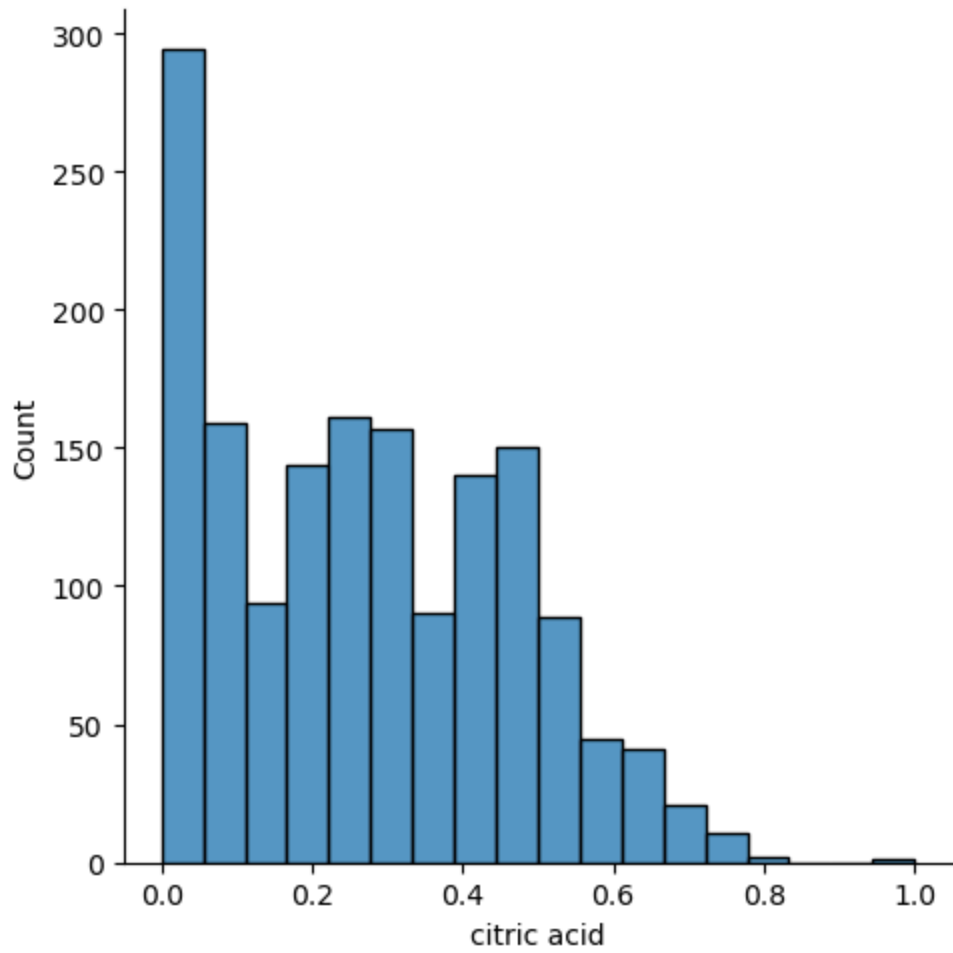
```
In [19]: print(df.columns)
```

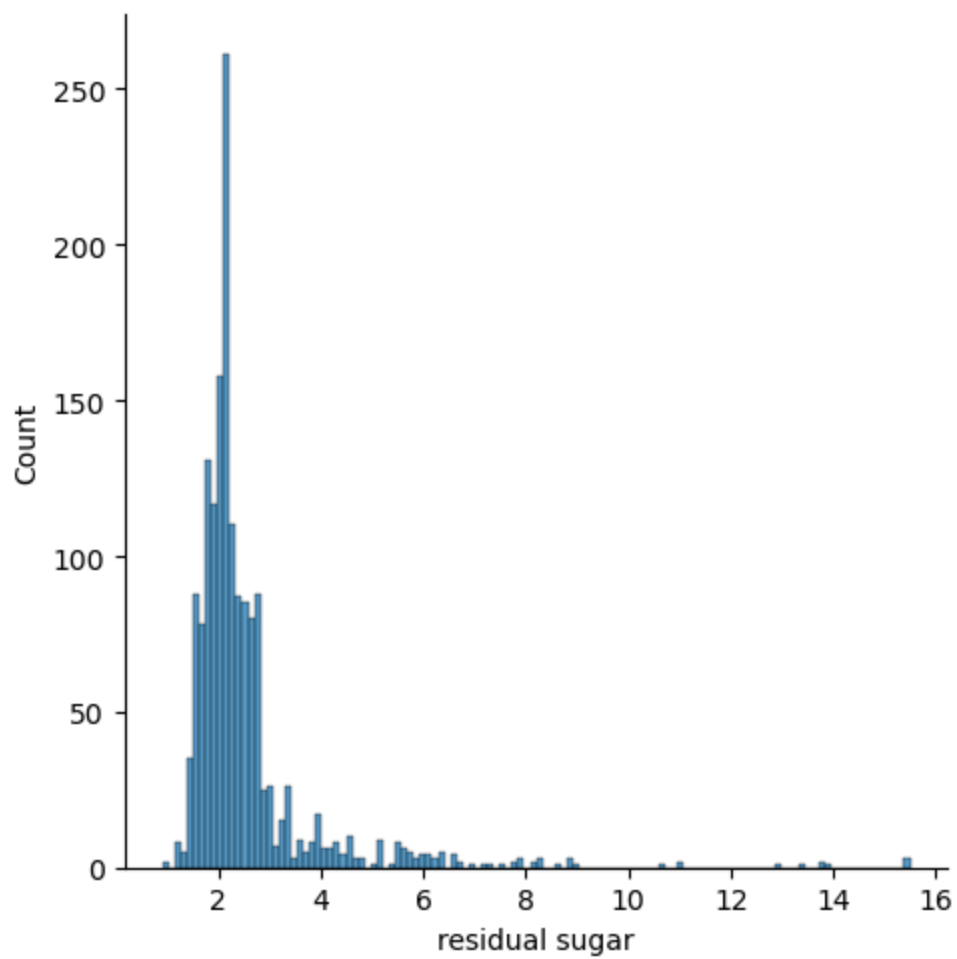
```
Index(['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar',
       'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'density',
       'pH', 'sulphates', 'alcohol', 'quality'],
      dtype='object')
```

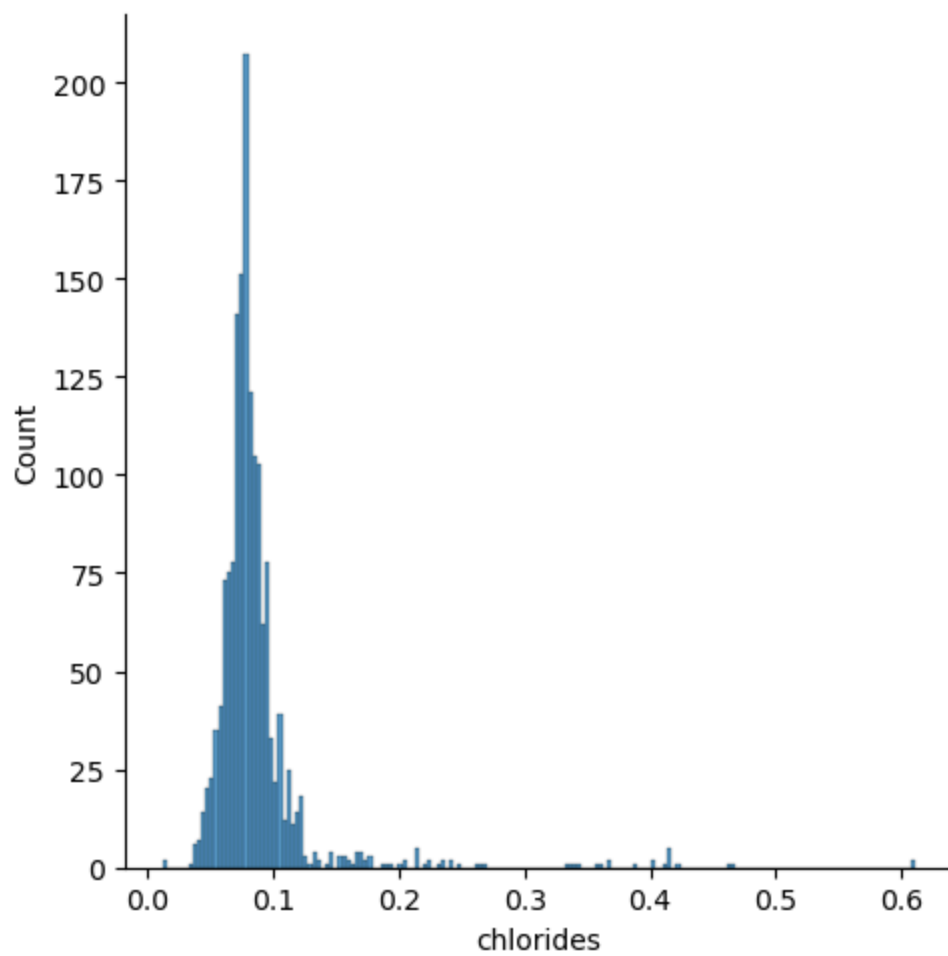
```
In [20]: sns.displot(data=df, x="citric acid")
plt.show
sns.displot(data=df, x="residual sugar")
plt.show
sns.displot(data=df, x="chlorides")
plt.show
sns.displot(data=df, x="free sulfur dioxide")
plt.show
sns.displot(data=df, x="total sulfur dioxide")
plt.show
```

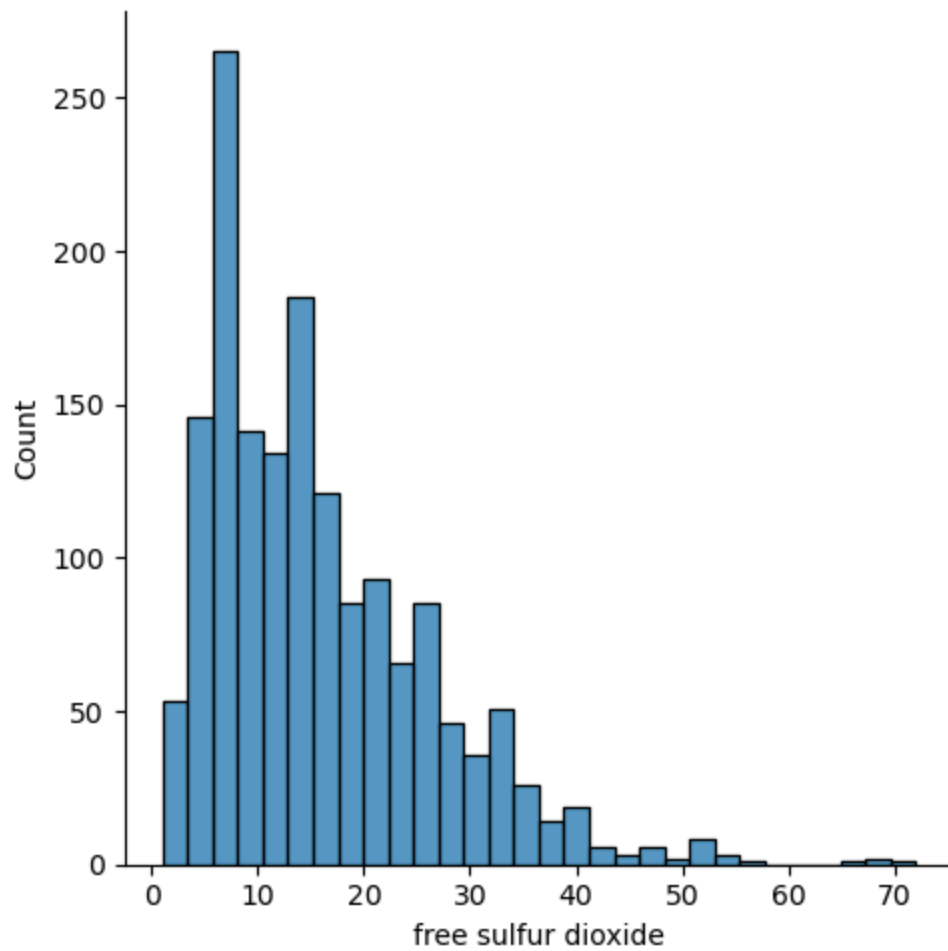
```
sns.displot(data=df, x="alcohol")  
plt.show
```

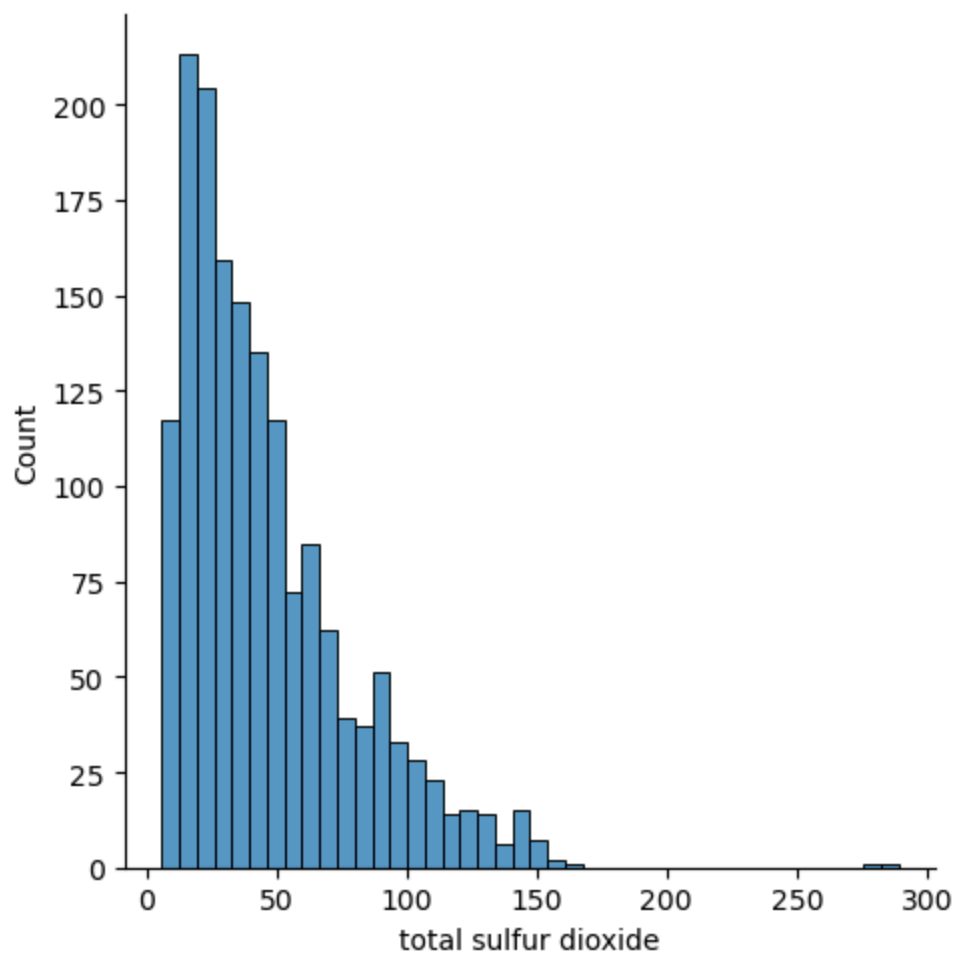
Out[20]: <function matplotlib.pyplot.show(close=None, block=None)>

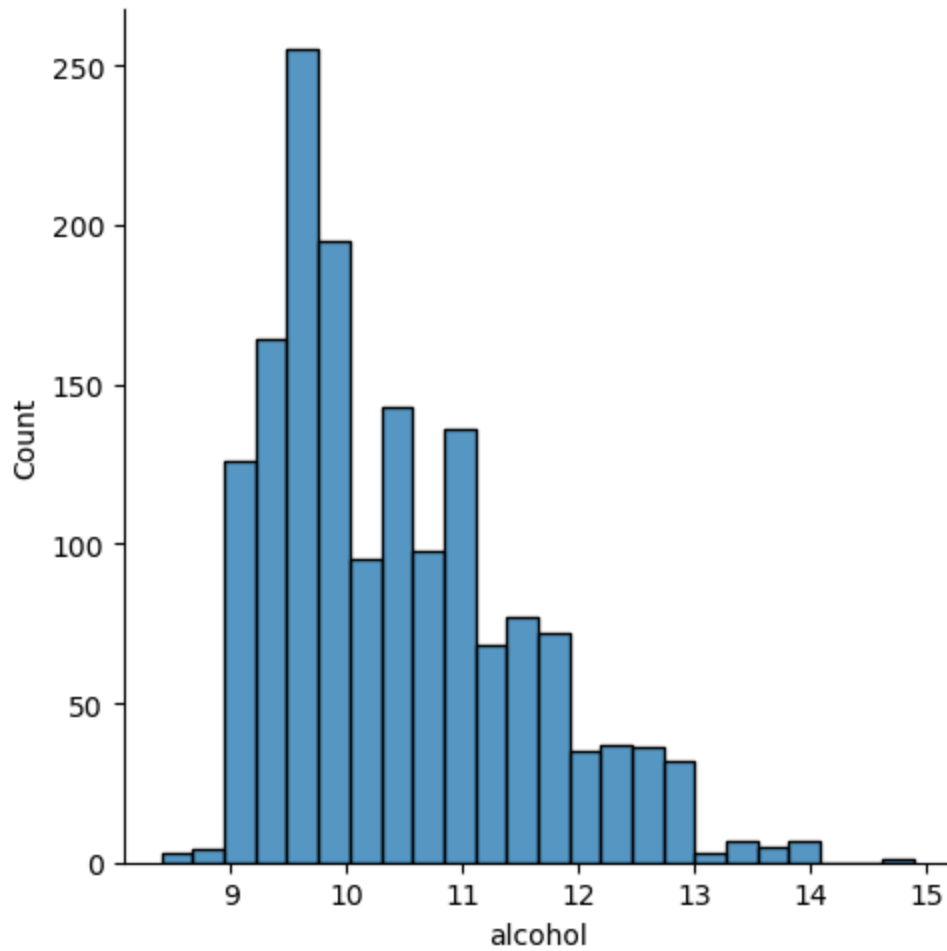






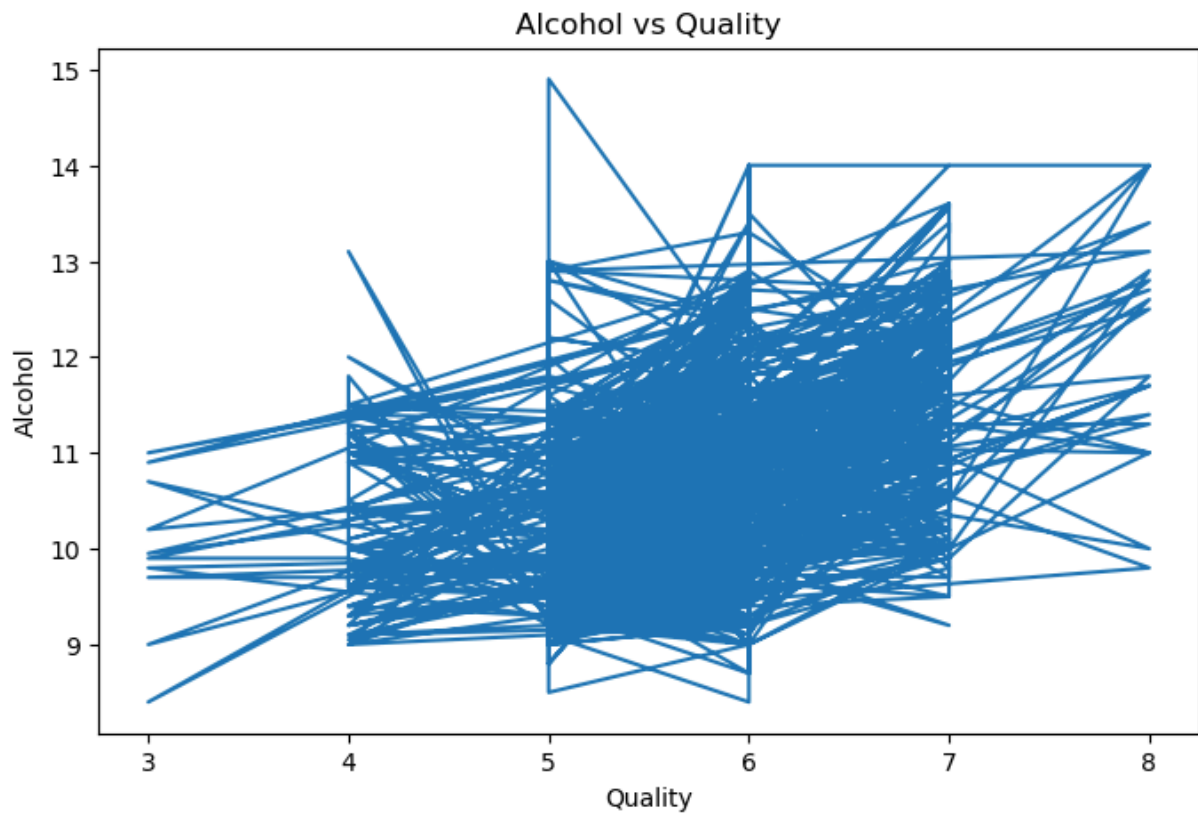






Bivariate Analysis (Line Chart)

```
In [21]: plt.figure(figsize=(8, 5))
plt.plot(df["quality"], df["alcohol"])
plt.xlabel("Quality")
plt.ylabel("Alcohol")
plt.title("Alcohol vs Quality")
plt.show()
```



Bivariate Analysis (Pie Chart)

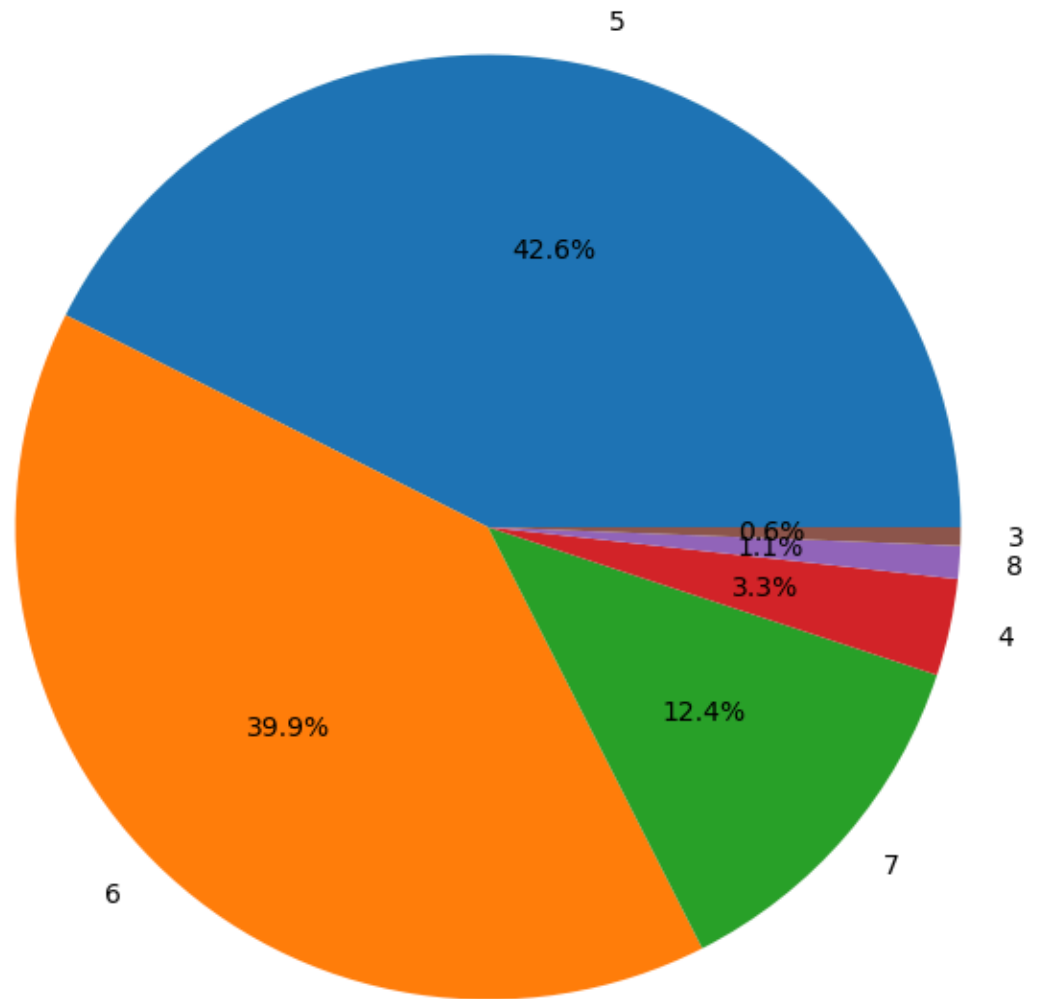
```
In [22]: quality_counts = df["quality"].value_counts()

plt.figure(figsize=(8, 8))

plt.pie(
    quality_counts.values,
    labels=quality_counts.index,
    autopct="%1.1f%%"
)

plt.title("Wine Quality Distribution")
plt.show()
```

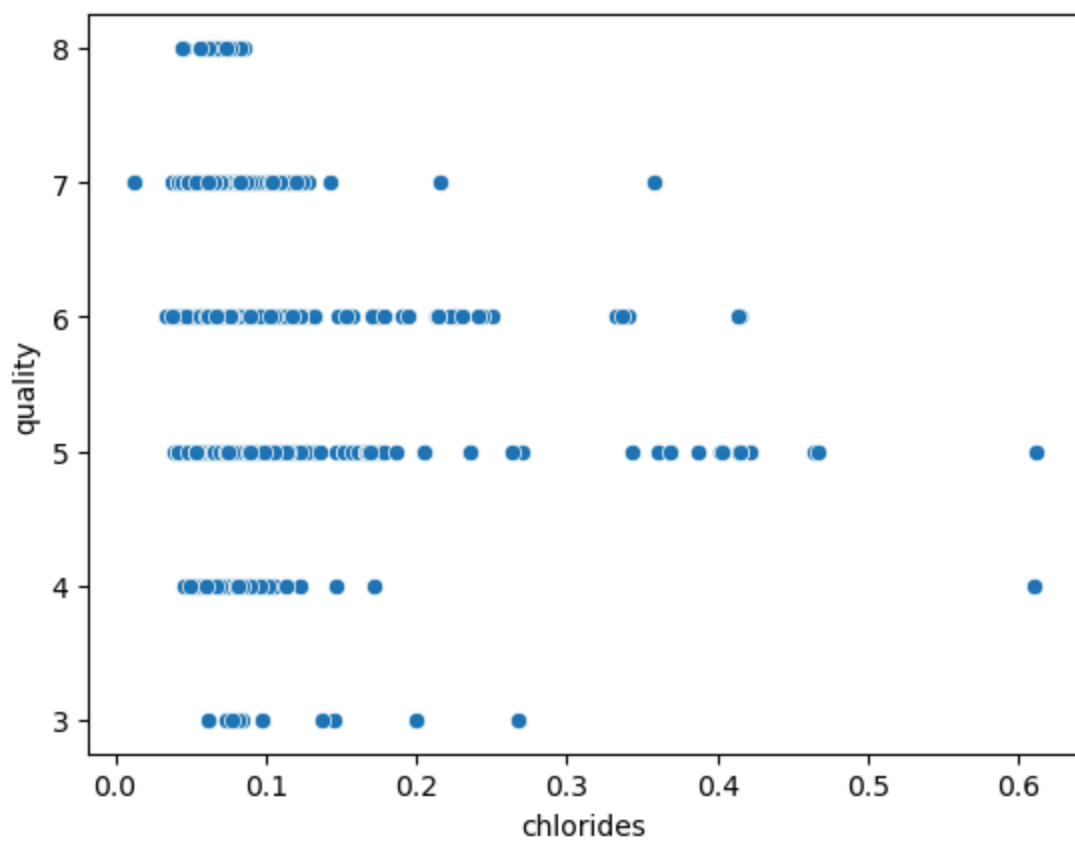
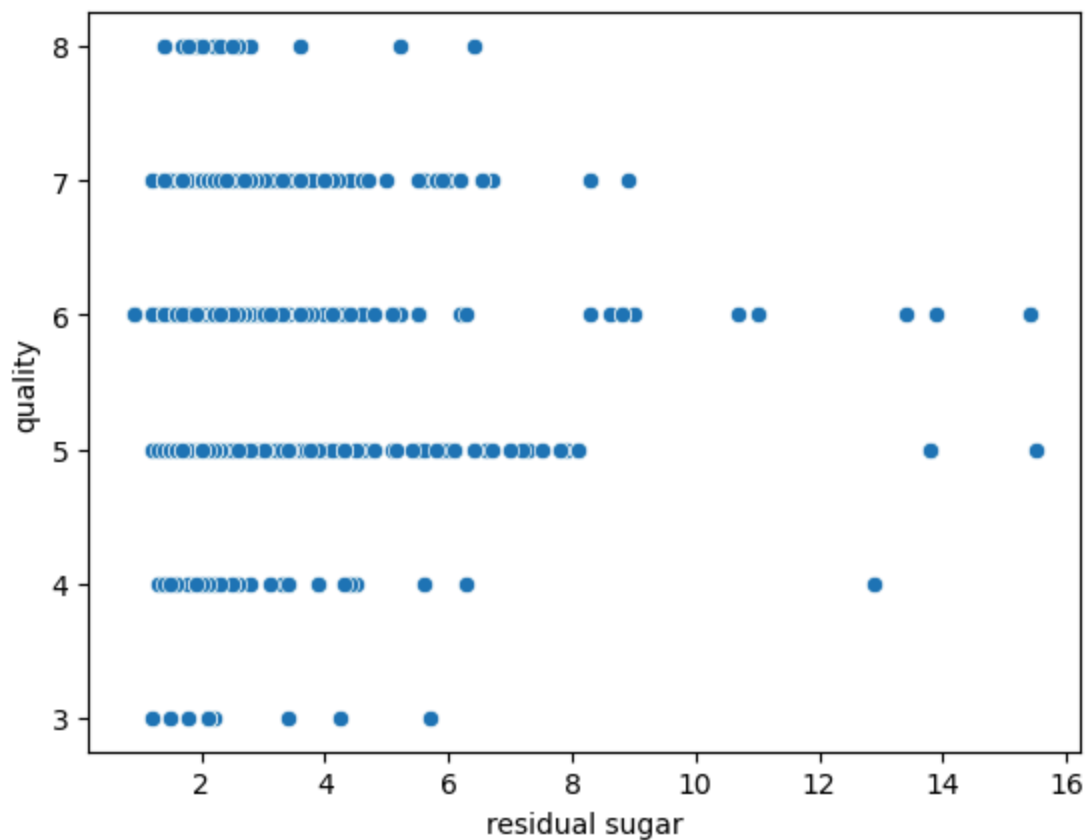
Wine Quality Distribution

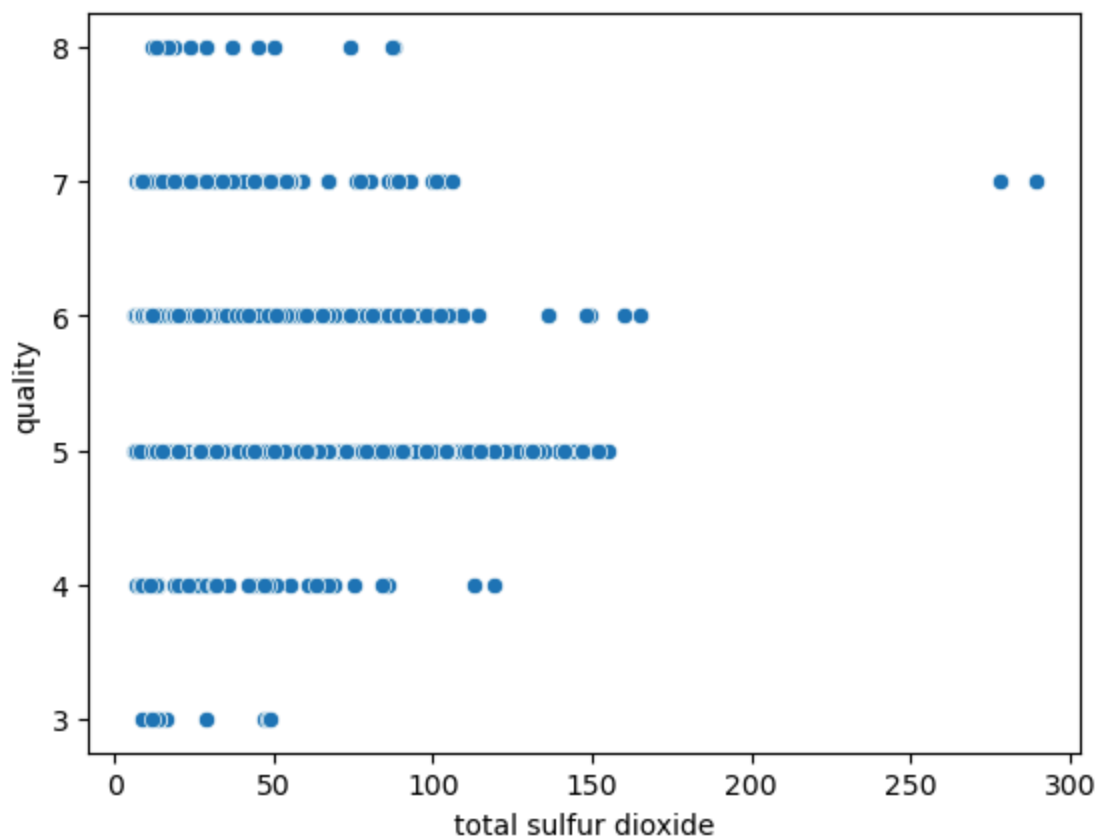
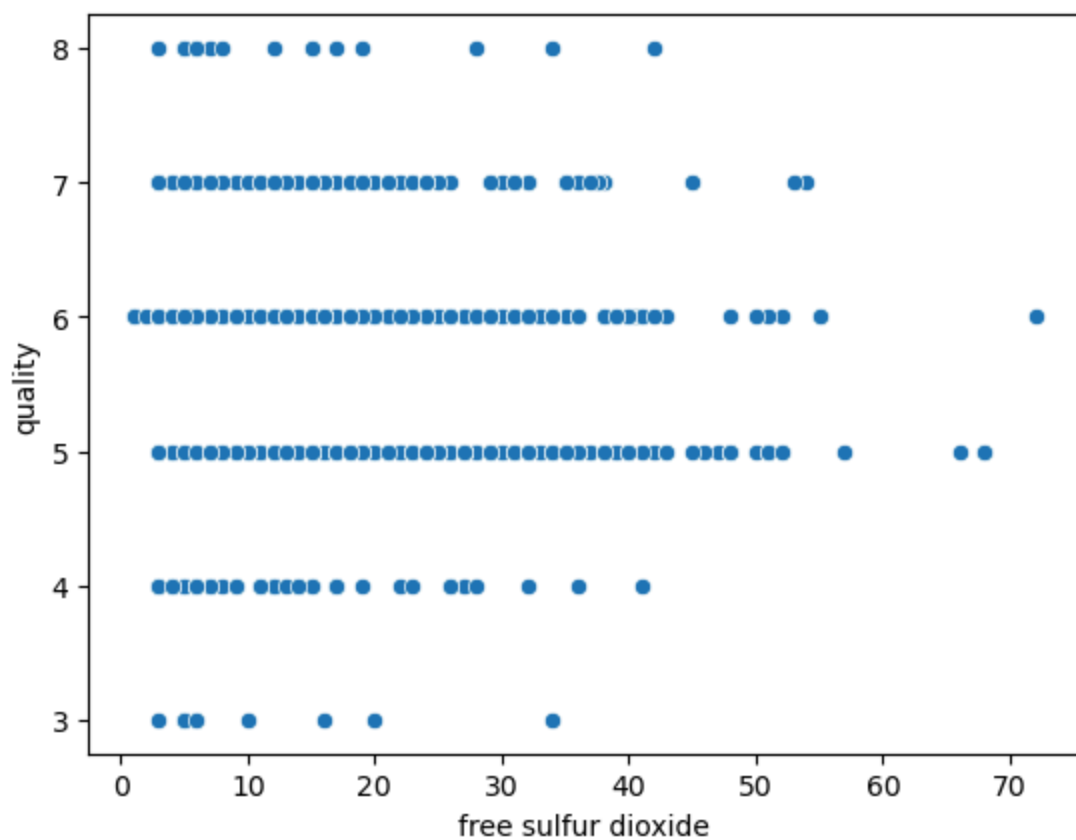


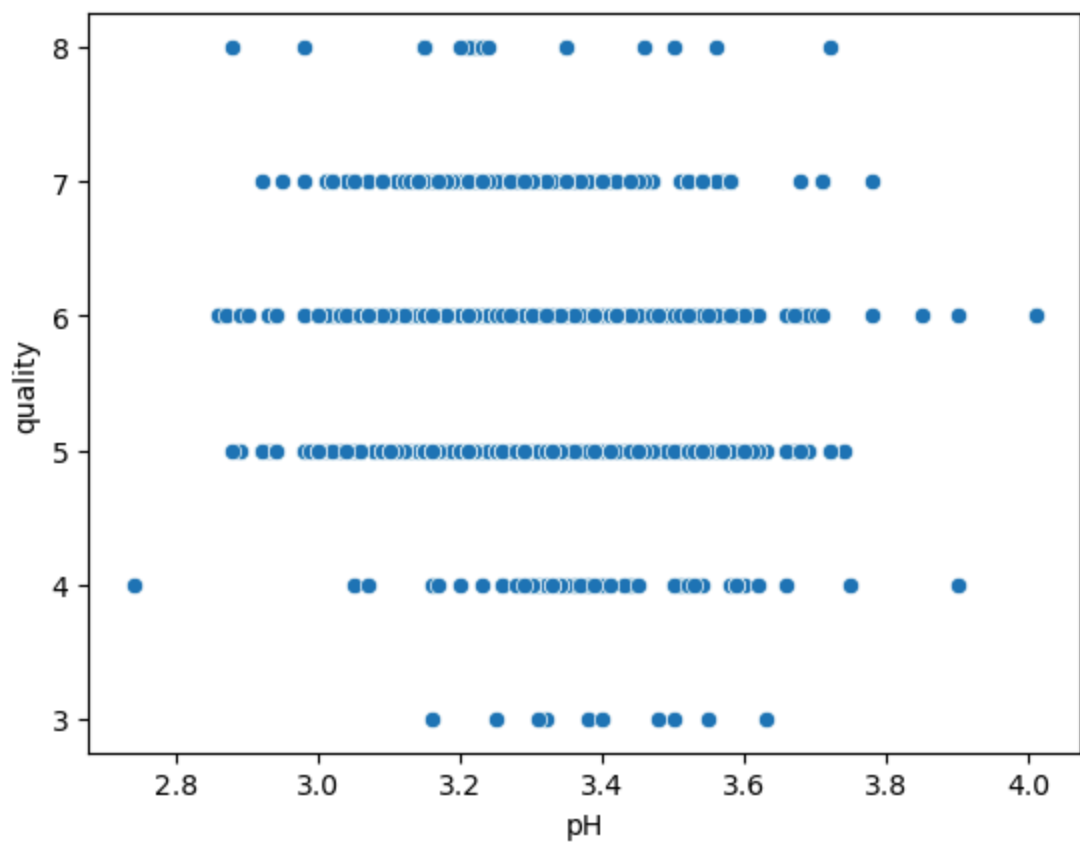
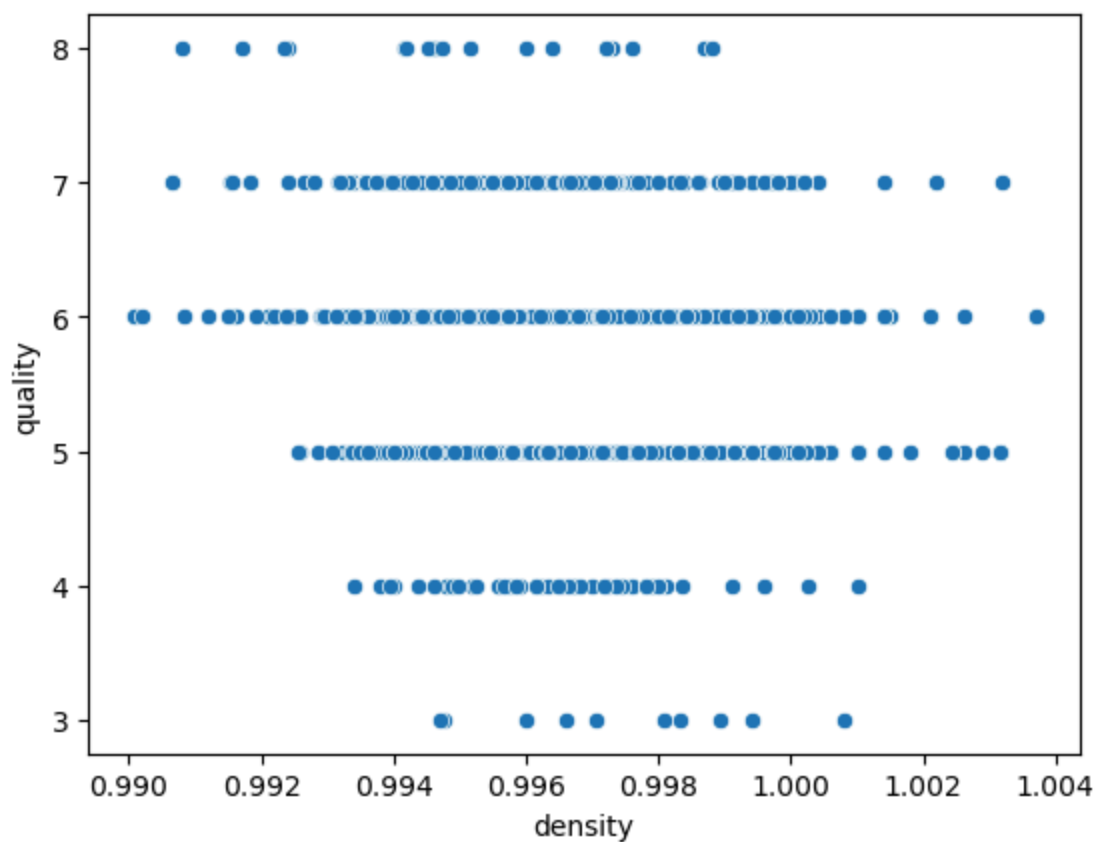
Bivariate Analysis (Scatter Plot)

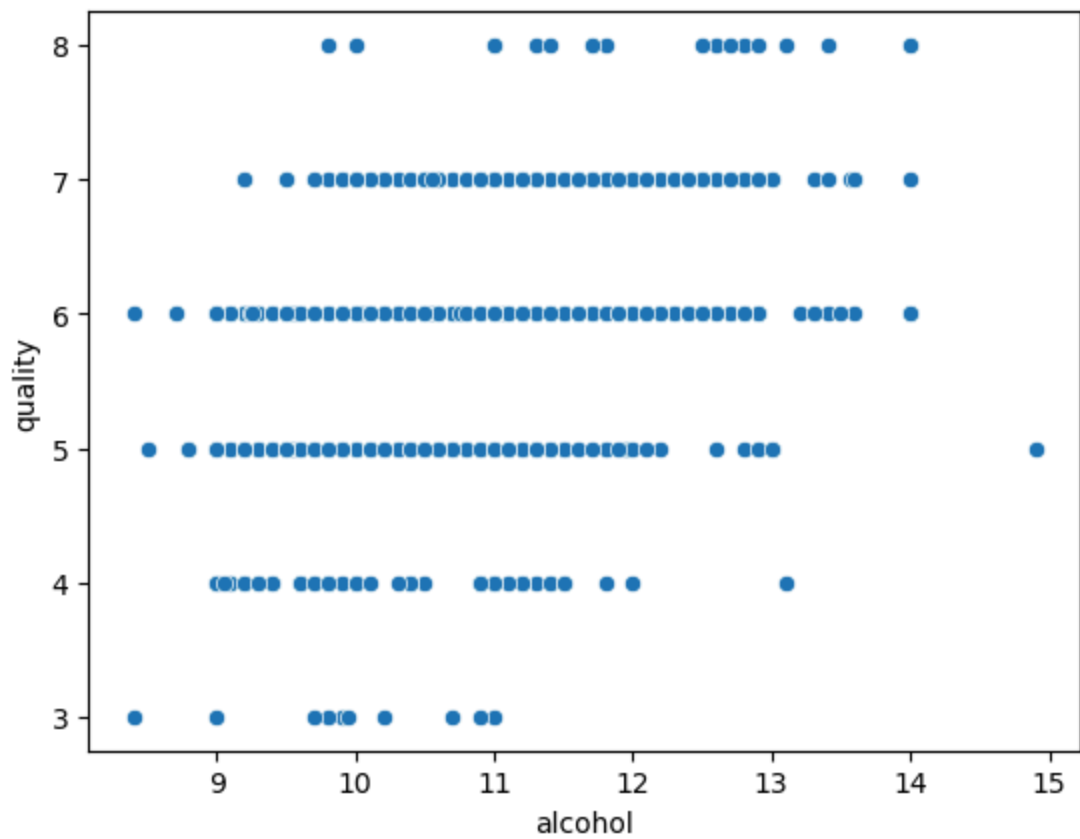
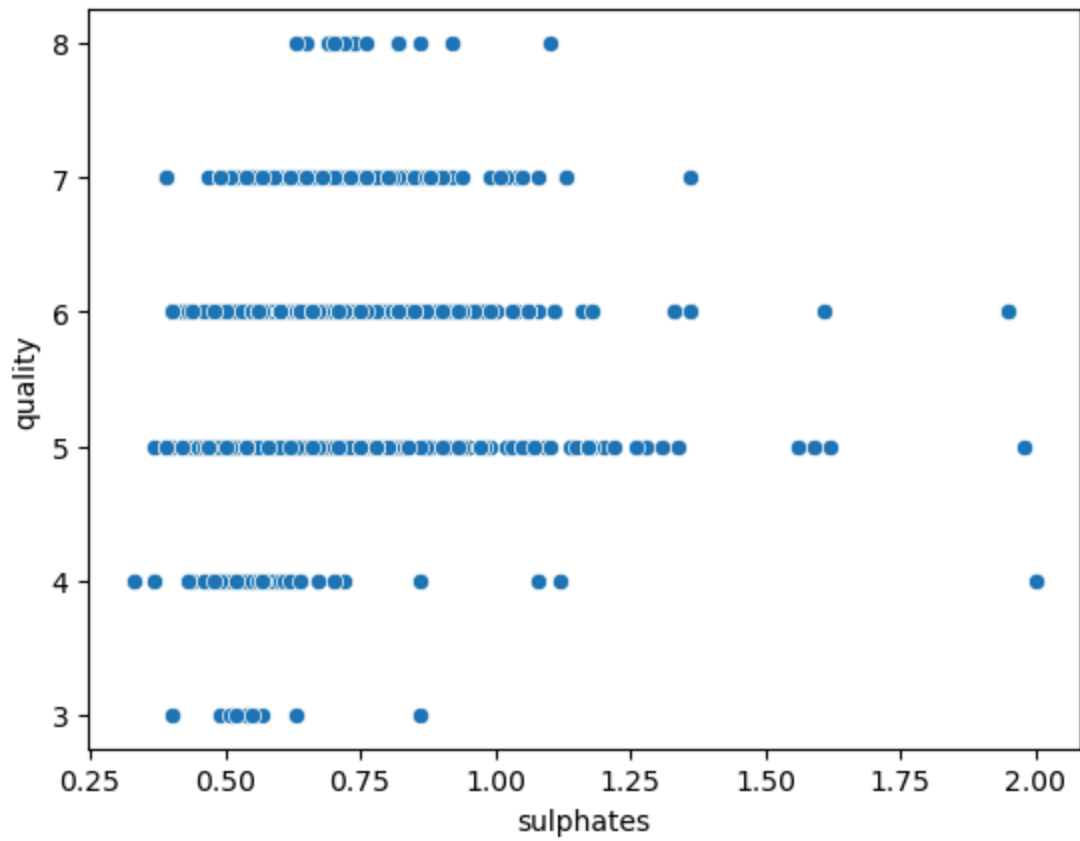
```
In [23]: sns.scatterplot(x="residual sugar",y="quality",data=df)
plt.show()
sns.scatterplot(x="chlorides",y="quality",data=df)
plt.show()
sns.scatterplot(x="free sulfur dioxide",y="quality",data=df)
plt.show()
sns.scatterplot(x="total sulfur dioxide",y="quality",data=df)
plt.show()
sns.scatterplot(x="density",y="quality",data=df)
plt.show()
sns.scatterplot(x="pH",y="quality",data=df)
plt.show()
sns.scatterplot(x="sulphates",y="quality",data=df)
plt.show()
```

```
sns.scatterplot(x="alcohol",y="quality",data=df)  
plt.show()
```



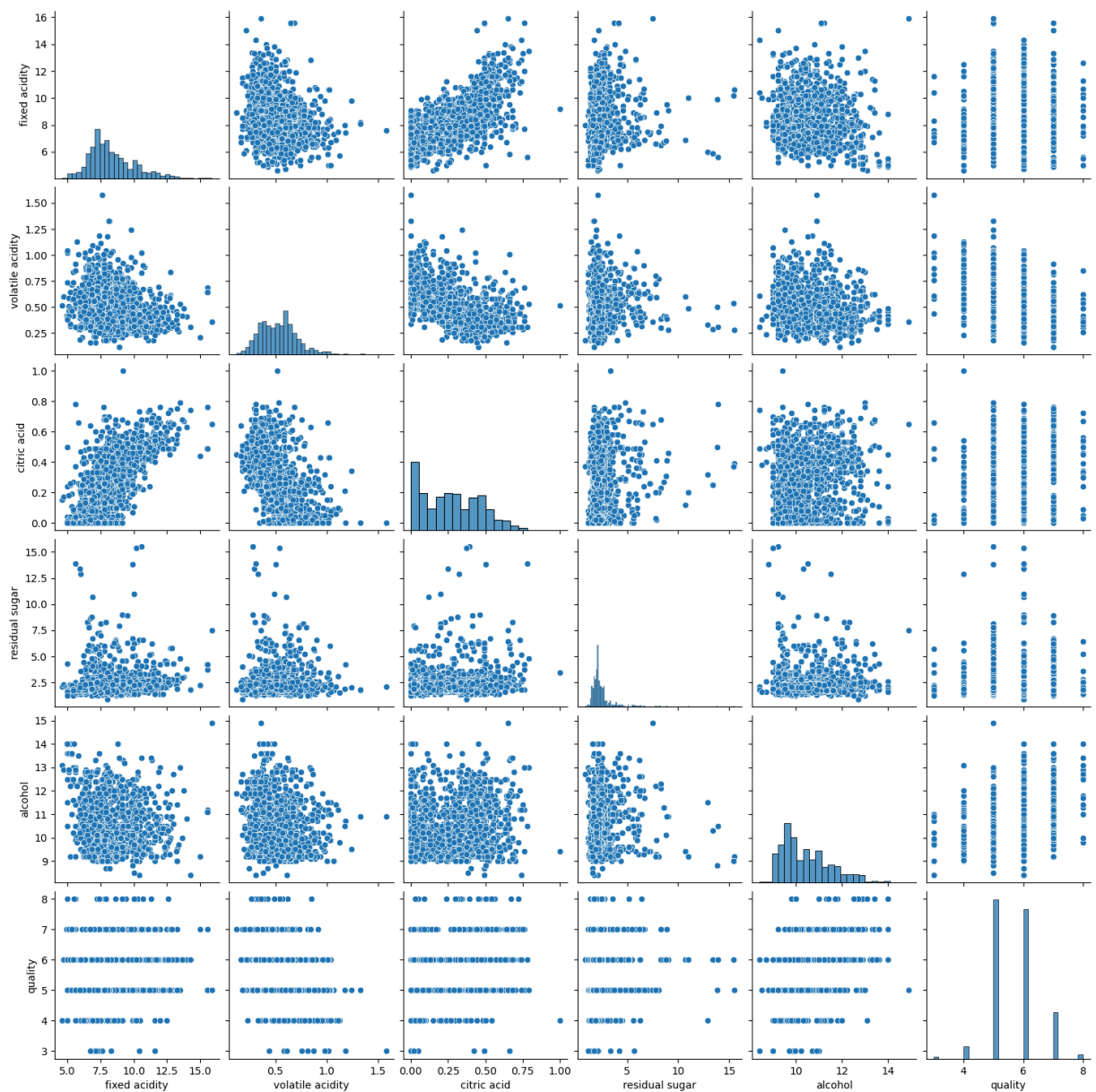






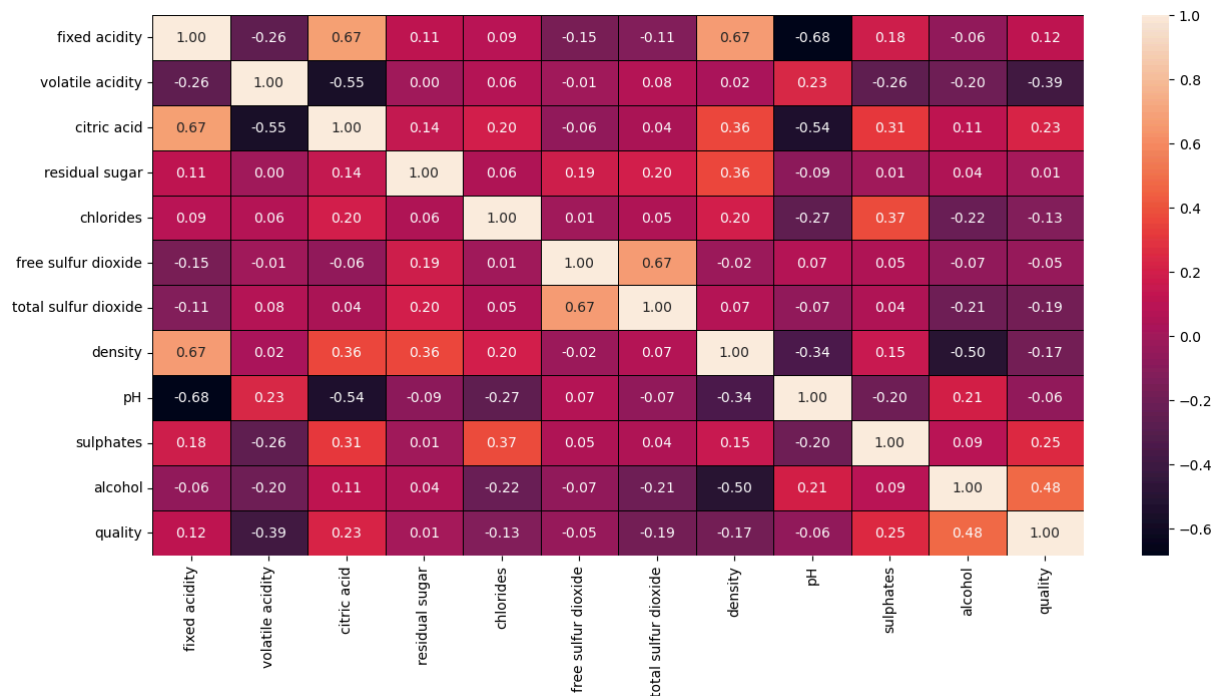
Multivariate Analysis (Pair Plot)

```
In [24]: sns.pairplot(
          df[["fixed acidity", "volatile acidity", "citric acid",
              "residual sugar", "alcohol", "quality"]]
        )
plt.show()
```



```
In [25]: plt.figure(figsize=(15,7))
          sns.heatmap(df.corr(),annot=True,linewidths=0.5,linecolor="black",fmt='.2f')
```

Out[25]: <Axes: >



Decision Tree Classification Algorithm

```
In [34]: from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, co
from sklearn.inspection import permutation_importance
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)
```

```
Out[34]: DecisionTreeClassifier
DecisionTreeClassifier(random_state=42)
```

```
In [27]: X = df.drop('quality', axis = 1)
y=df['quality']
```

```
In [28]: X_train, X_test, y_train, y_test = train_test_split(
X,
y,
test_size = 0.20,
random_state = 42,
stratify=y
)
results=[]
```

```
In [29]: def evaluate(name,model):
model.fit(X_train,y_train)
pred=model.predict(X_test)
acc=accuracy_score(y_test,pred)
print(name,acc)
print(classification_report(y_test,pred))
```

```

cm = confusion_matrix(y_test,pred)
sns.heatmap(cm,annot=True,fmt='d',cmap='Blues')
plt.title(name)
plt.show()
results.append([name, acc, model.get_depth(), model.tree_.node_count])
return model

```

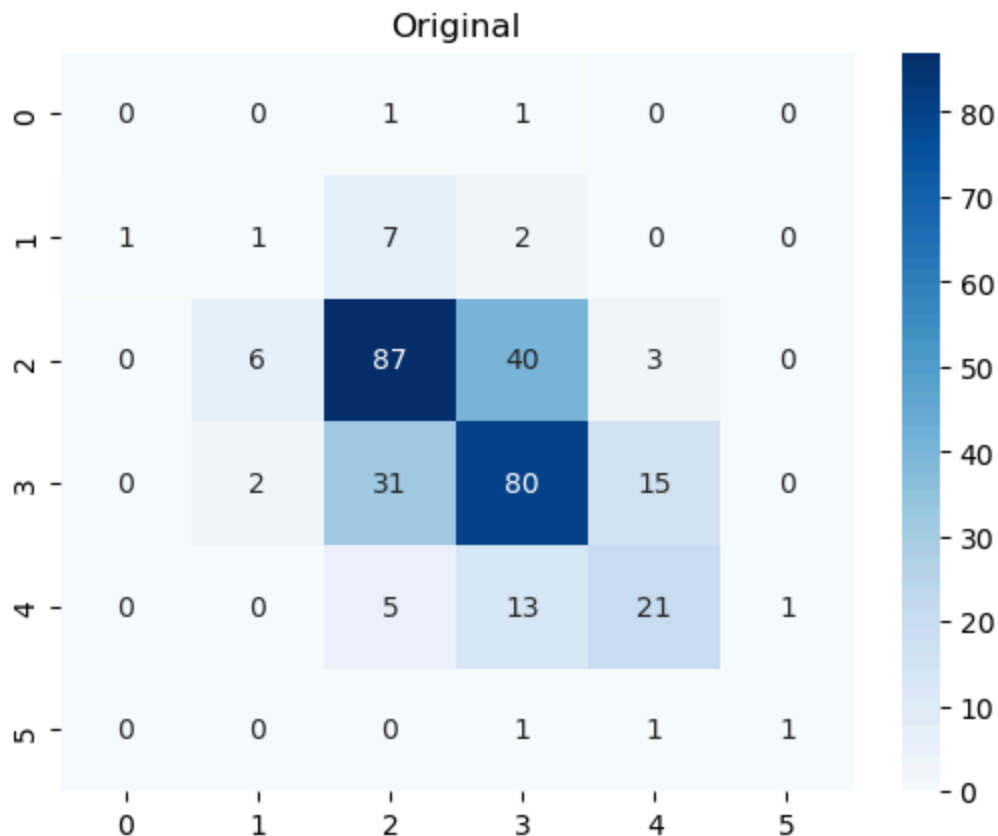
```

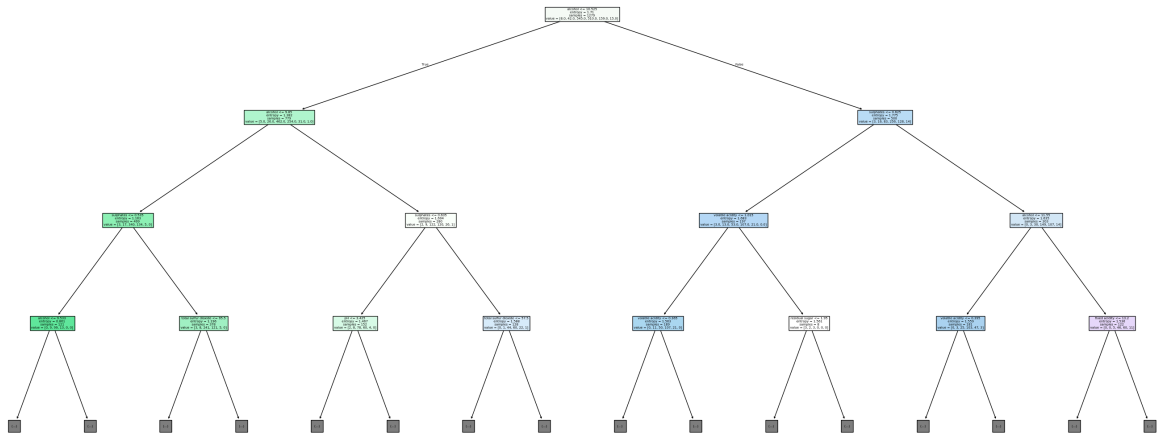
In [30]: tree=evaluate('Original',DecisionTreeClassifier(criterion='entropy',random_state=42)
plt.figure(figsize=(35,15))
plot_tree(tree,feature_names=X.columns,filled=True,max_depth=3)
plt.show()

```

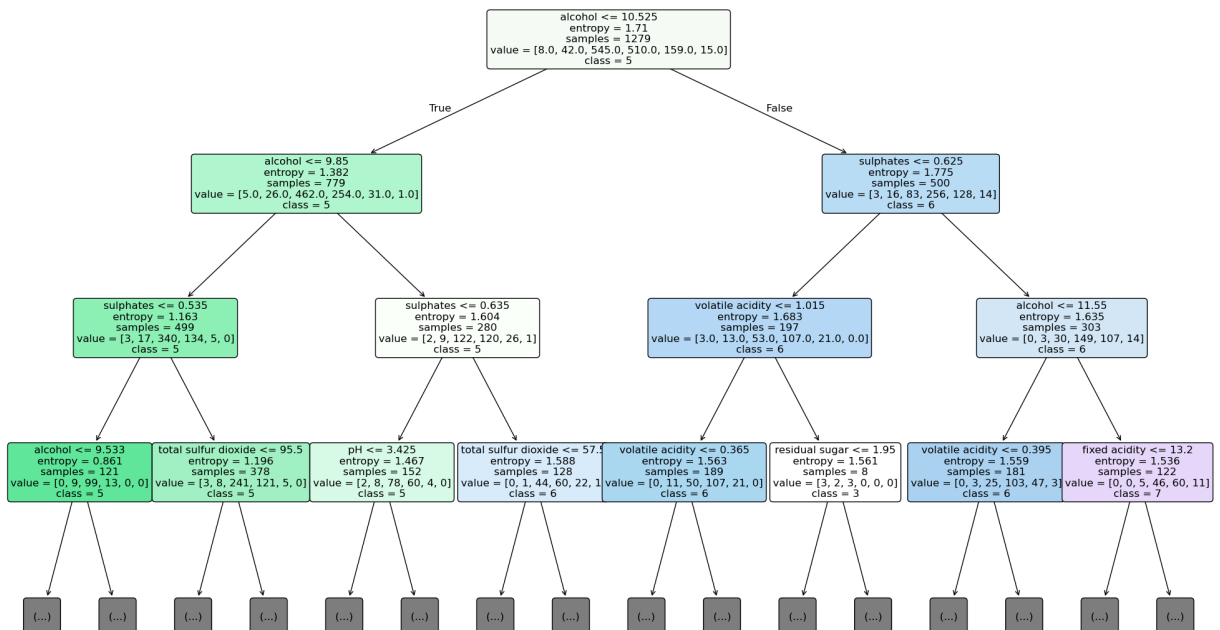
Original 0.59375

	precision	recall	f1-score	support
3	0.00	0.00	0.00	2
4	0.11	0.09	0.10	11
5	0.66	0.64	0.65	136
6	0.58	0.62	0.60	128
7	0.53	0.53	0.53	40
8	0.50	0.33	0.40	3
accuracy			0.59	320
macro avg	0.40	0.37	0.38	320
weighted avg	0.59	0.59	0.59	320





```
In [31]: plt.figure(figsize=(25,15))
plot_tree(
    tree,
    feature_names=X.columns,
    class_names=[str(i) for i in sorted(y.unique())],
    filled=True,
    rounded=True,
    fontsize=12,
    max_depth=3
)
plt.show()
```

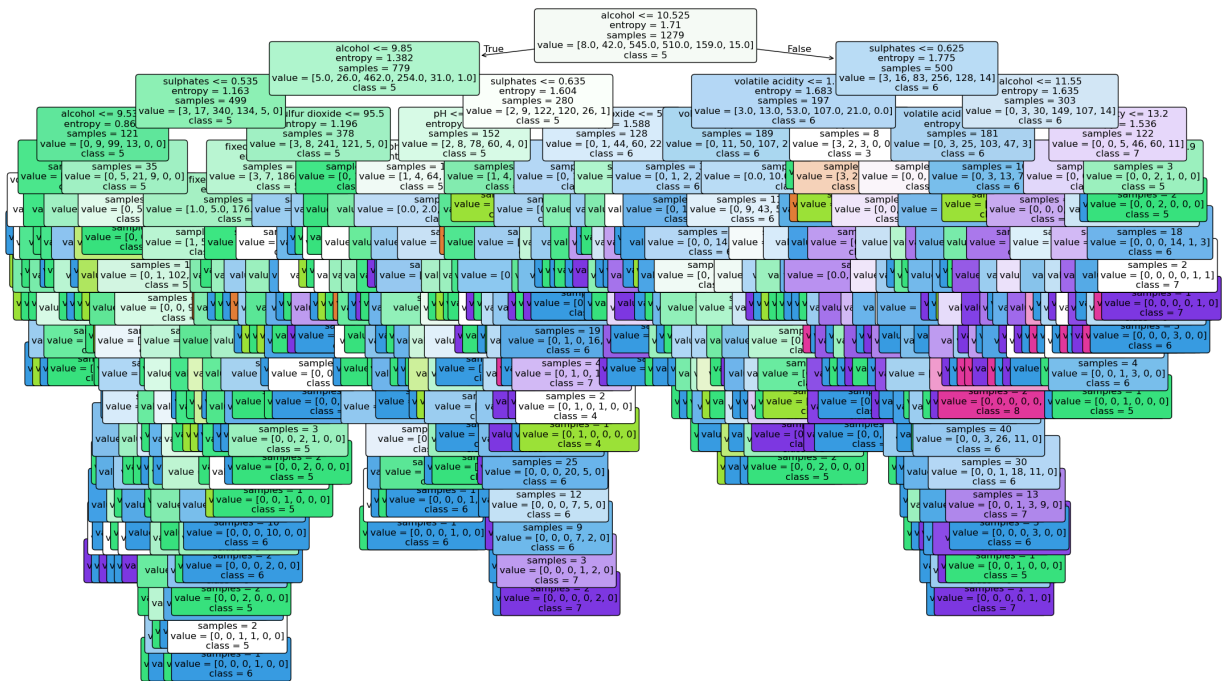


```
In [32]: plt.figure(figsize=(25,15))
plot_tree(
    tree,
    feature_names=X.columns,
    class_names=[str(i) for i in sorted(y.unique())],
```

```

filled=True,
rounded=True,
fontsize=12,
max_depth=20
)
plt.show()

```



```

In [37]: importance = pd.Series(
          dt_model.feature_importances_, index=X.columns).sort_values(ascending=False)
          print(importance)

```

```

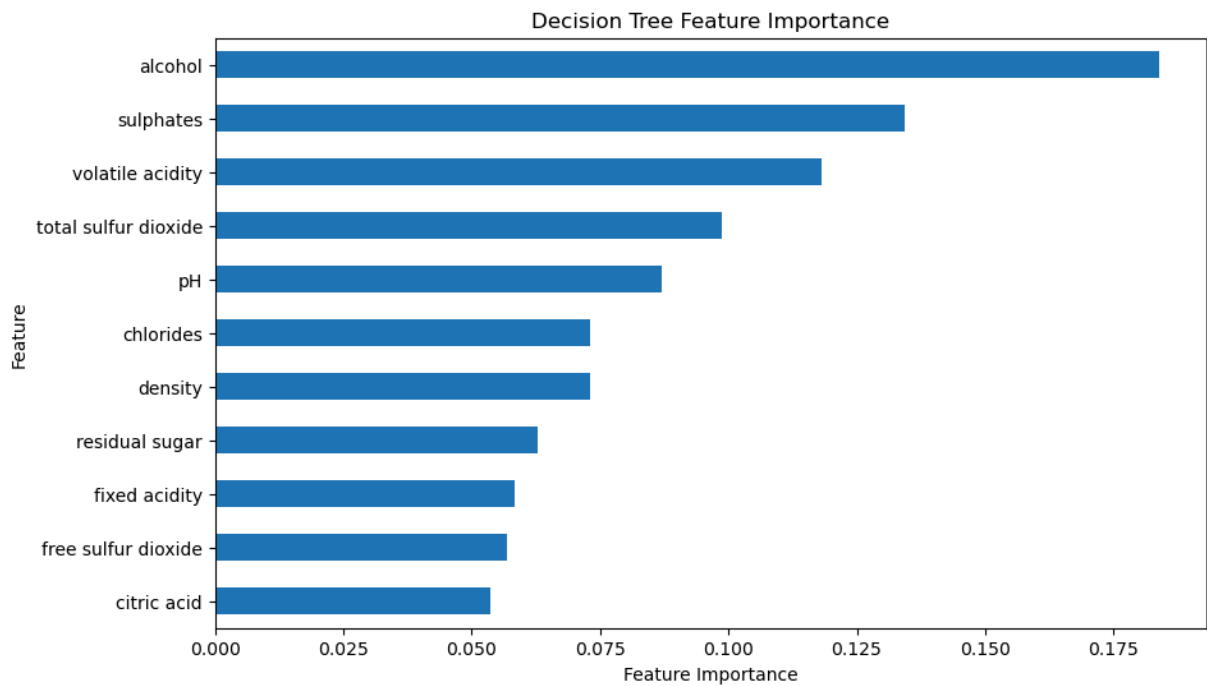
alcohol      0.183838
sulphates    0.134306
volatile acidity 0.118186
total sulfur dioxide 0.098721
pH           0.087091
chlorides    0.073129
density      0.073024
residual sugar 0.062869
fixed acidity 0.058263
free sulfur dioxide 0.056880
citric acid  0.053693
dtype: float64

```

```

In [40]: plt.figure(figsize=(10, 6))
          importance.sort_values().plot(kind='barh')
          plt.xlabel('Feature Importance')
          plt.ylabel('Feature')
          plt.title('Decision Tree Feature Importance')
          plt.show()

```



```
In [42]: train_accuracy = accuracy_score(y_train, dt_model.predict(X_train))
test_accuracy = accuracy_score(y_test, dt_model.predict(X_test))
print("Training Accuracy:", train_accuracy)
print("Testing Accuracy:", test_accuracy)
```

Training Accuracy: 1.0
Testing Accuracy: 0.60625

```
In [43]: for d in [2, 3, 4, 5, None]:
          evaluate(f'MaxDepth-{d}', DecisionTreeClassifier(max_depth=d))
```

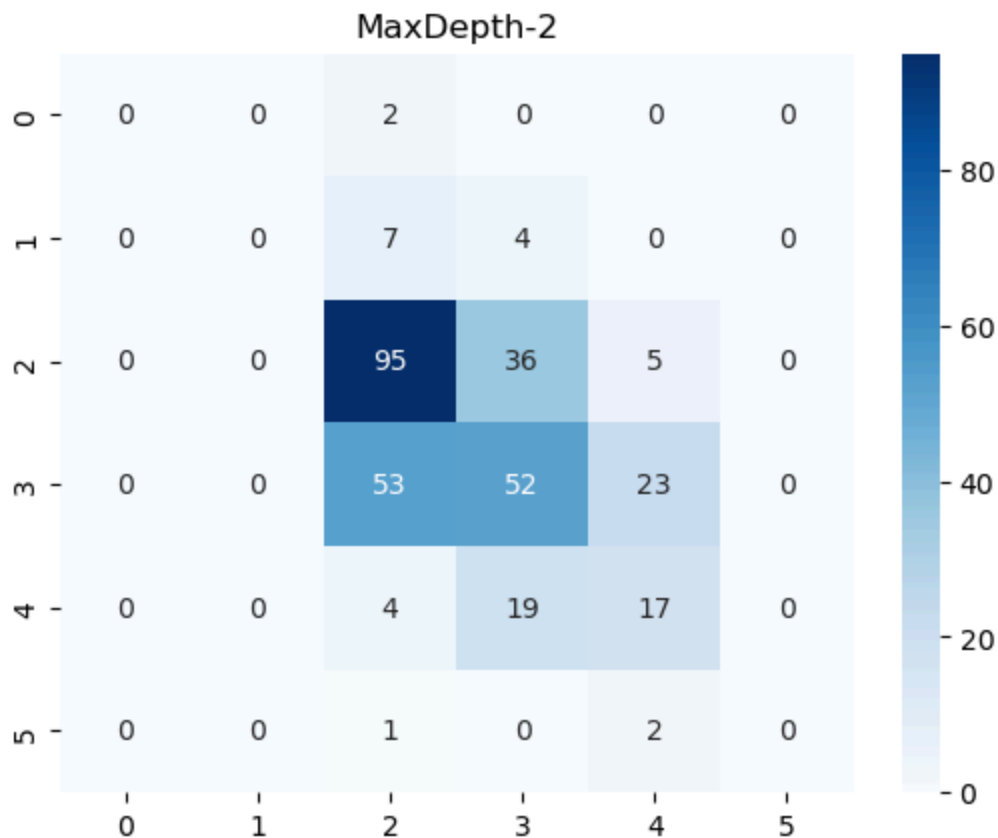
MaxDepth-2 0.5125

	precision	recall	f1-score	support
3	0.00	0.00	0.00	2
4	0.00	0.00	0.00	11
5	0.59	0.70	0.64	136
6	0.47	0.41	0.44	128
7	0.36	0.42	0.39	40
8	0.00	0.00	0.00	3
accuracy			0.51	320
macro avg	0.24	0.25	0.24	320
weighted avg	0.48	0.51	0.49	320

```

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with
no predicted samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with
no predicted samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with
no predicted samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```



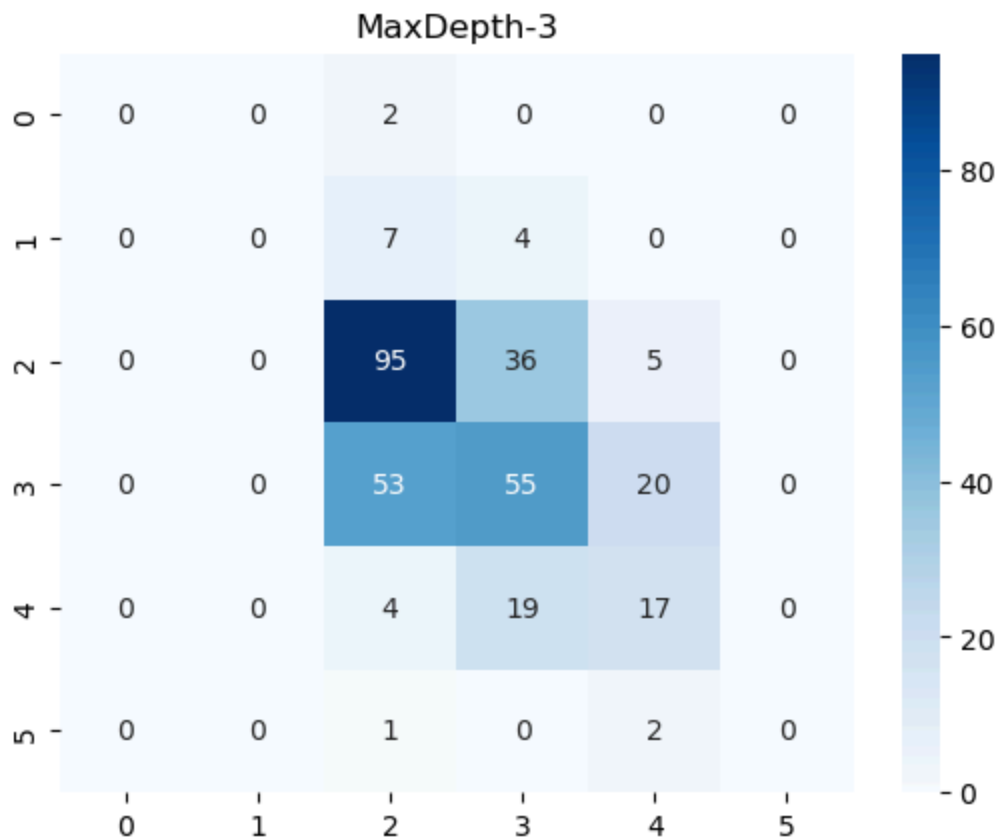
MaxDepth-3 0.521875

	precision	recall	f1-score	support
3	0.00	0.00	0.00	2
4	0.00	0.00	0.00	11
5	0.59	0.70	0.64	136
6	0.48	0.43	0.45	128
7	0.39	0.42	0.40	40
8	0.00	0.00	0.00	3
accuracy			0.52	320
macro avg	0.24	0.26	0.25	320
weighted avg	0.49	0.52	0.50	320

```

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with
no predicted samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with
no predicted samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with
no predicted samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

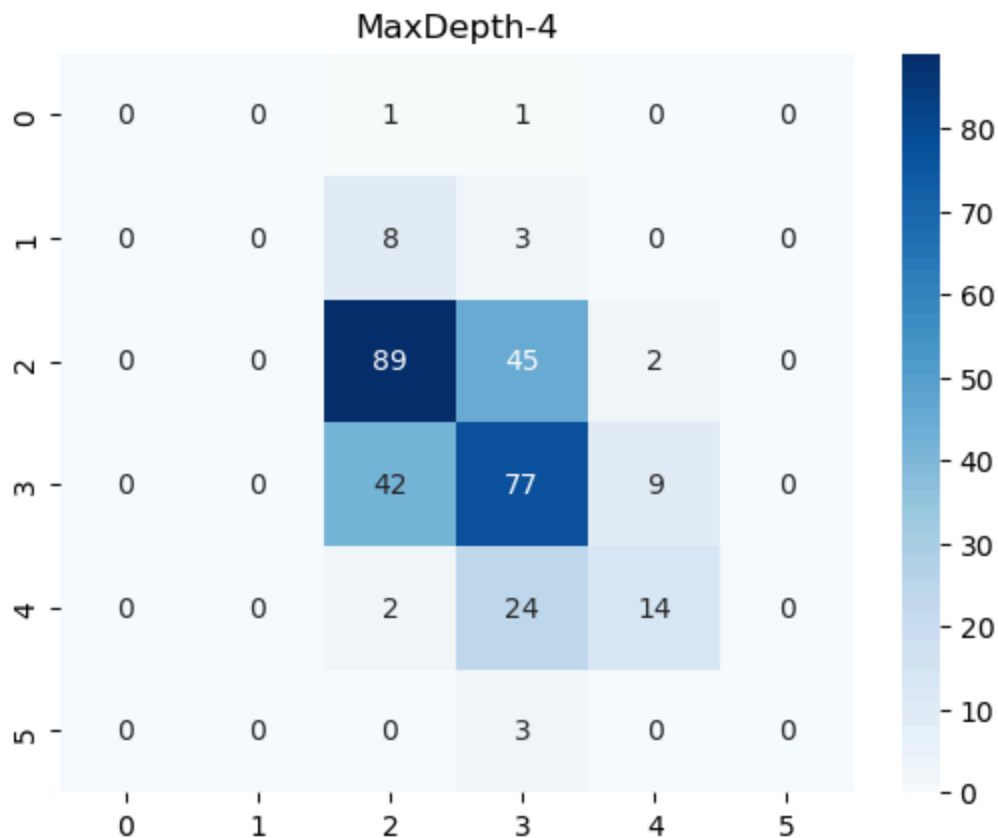
```



MaxDepth-4 0.5625

	precision	recall	f1-score	support
3	0.00	0.00	0.00	2
4	0.00	0.00	0.00	11
5	0.63	0.65	0.64	136
6	0.50	0.60	0.55	128
7	0.56	0.35	0.43	40
8	0.00	0.00	0.00	3
accuracy			0.56	320
macro avg	0.28	0.27	0.27	320
weighted avg	0.54	0.56	0.55	320

```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with
no predicted samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with
no predicted samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with
no predicted samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```



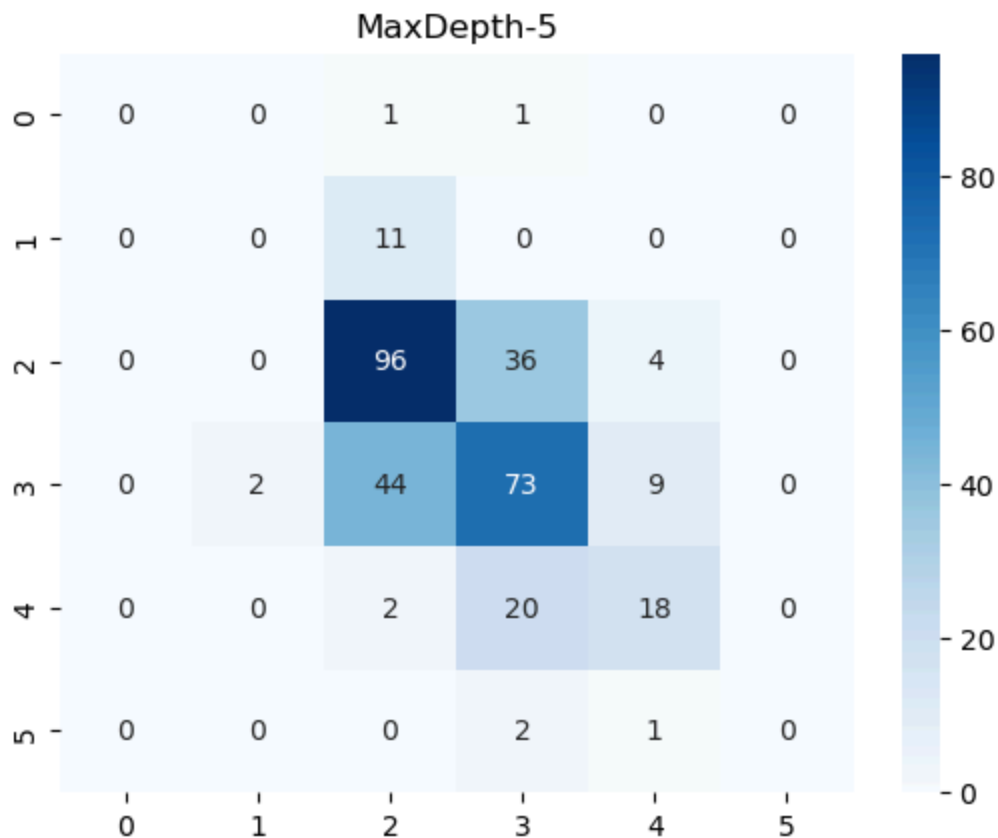
MaxDepth-5 0.584375

	precision	recall	f1-score	support
3	0.00	0.00	0.00	2
4	0.00	0.00	0.00	11
5	0.62	0.71	0.66	136
6	0.55	0.57	0.56	128
7	0.56	0.45	0.50	40
8	0.00	0.00	0.00	3
accuracy			0.58	320
macro avg	0.29	0.29	0.29	320
weighted avg	0.56	0.58	0.57	320

```

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with
no predicted samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with
no predicted samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with
no predicted samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

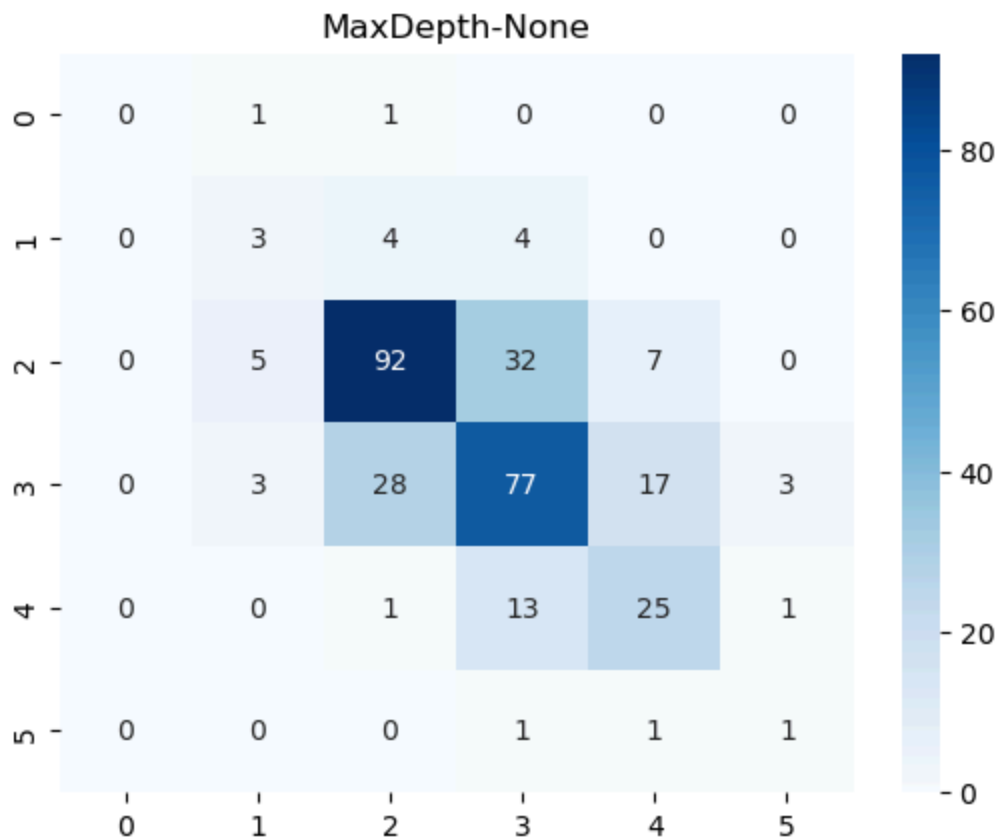
```



MaxDepth=None 0.61875

	precision	recall	f1-score	support
3	0.00	0.00	0.00	2
4	0.25	0.27	0.26	11
5	0.73	0.68	0.70	136
6	0.61	0.60	0.60	128
7	0.50	0.62	0.56	40
8	0.20	0.33	0.25	3
accuracy			0.62	320
macro avg	0.38	0.42	0.40	320
weighted avg	0.63	0.62	0.62	320

```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with
no predicted samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with
no predicted samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with
no predicted samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```



Hyperparameter Tuning

```
In [45]: from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeClassifier
param_grid = {
    'criterion': ['entropy'],
    'max_depth': [3, 5, 7, 10, None],
    'min_samples_split': [2, 5, 10, 20],
    'min_samples_leaf': [1, 2, 4, 8],
    'max_features': [None, 'sqrt', 'log2'],
    'ccp_alpha': [0, 0.001, 0.005, 0.01]
}

grid = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_grid,
```

```

cv=5,
scoring='accuracy',
n_jobs=-1
)
grid.fit(X_train, y_train)
print("Best Parameters")
print(grid.best_params_)

print("\nBest Cross Validation Accuracy")
print(grid.best_score_)
best_tree = grid.best_estimator_

```

Best Parameters

```
{'ccp_alpha': 0, 'criterion': 'entropy', 'max_depth': 10, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 2}
```

Best Cross Validation Accuracy

0.6012500000000001

```
In [46]: y_pred = best_tree.predict(X_test)
print(classification_report(y_test,y_pred))
print("Accuracy=", accuracy_score(y_test,y_pred))
```

	precision	recall	f1-score	support
3	0.00	0.00	0.00	2
4	0.00	0.00	0.00	11
5	0.62	0.62	0.62	136
6	0.56	0.55	0.56	128
7	0.46	0.53	0.49	40
8	0.00	0.00	0.00	3
accuracy			0.55	320
macro avg	0.27	0.28	0.28	320
weighted avg	0.55	0.55	0.55	320

Accuracy= 0.546875

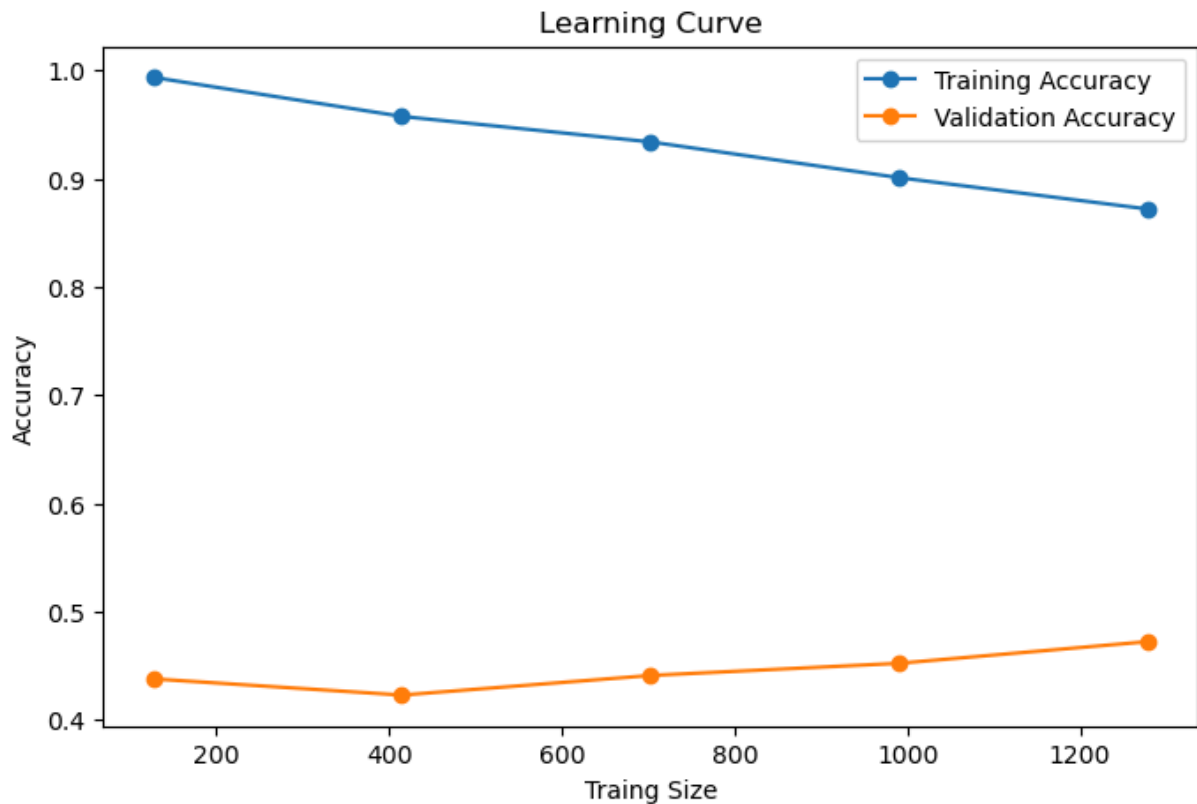
Cross Validation

```
In [48]: from sklearn.model_selection import cross_val_score
scores = cross_val_score(best_tree,X,y,cv=10,scoring='accuracy')
print(scores)
print("Average Accuracy=", scores.mean())
print("Std=", scores.std())
```

```
[0.525    0.5625   0.41875  0.475     0.4875   0.5375
 0.4625   0.4625   0.50625  0.46540881]
Average Accuracy= 0.4902908805031447
Std= 0.040528700639559496
```

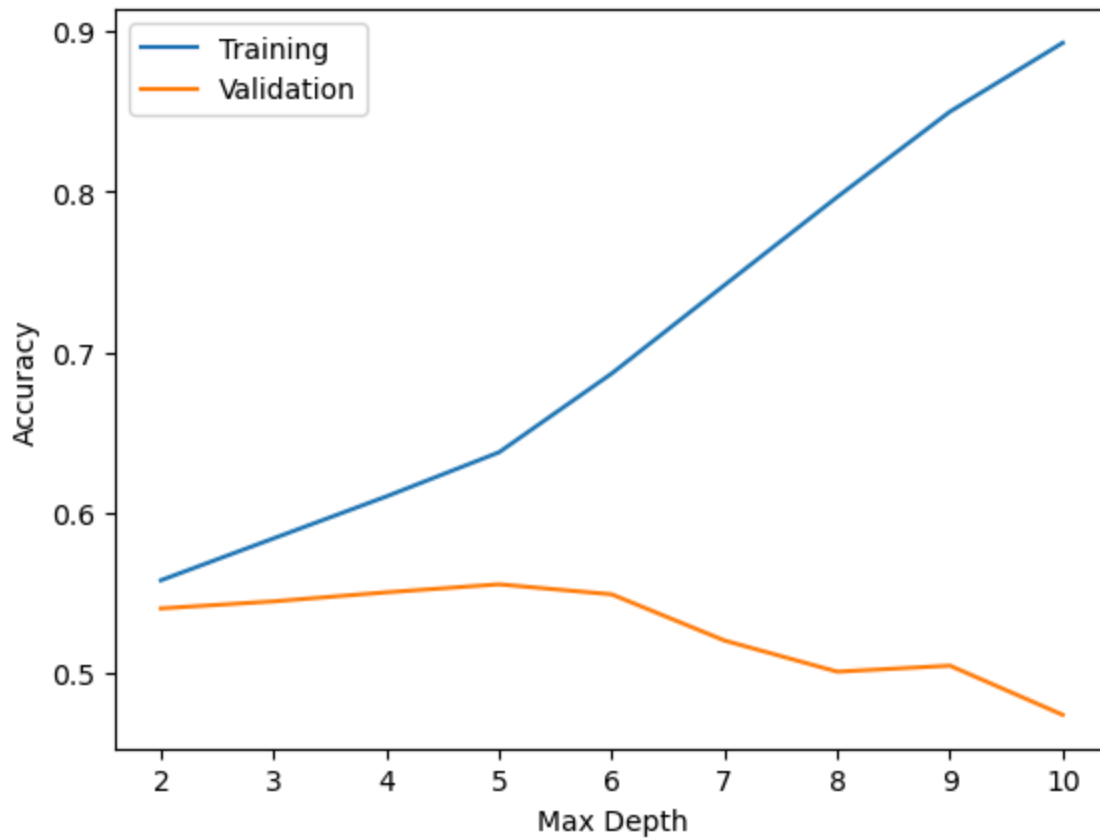
Learning Curve

```
In [57]: from sklearn.model_selection import learning_curve
train_Sizes,train_Scores,test_scores = learning_curve(
    best_tree,X,y,cv=5,scoring='accuracy')
plt.figure(figsize=(8,5))
plt.plot(train_Sizes,train_Scores.mean(axis=1),marker='o',label='Training Accuracy')
plt.plot(train_Sizes,test_scores.mean(axis=1),marker='o',label='Validation Accuracy')
plt.xlabel("Traing Size")
plt.ylabel("Accuracy")
plt.title("Learning Curve")
plt.legend()
plt.show()
```



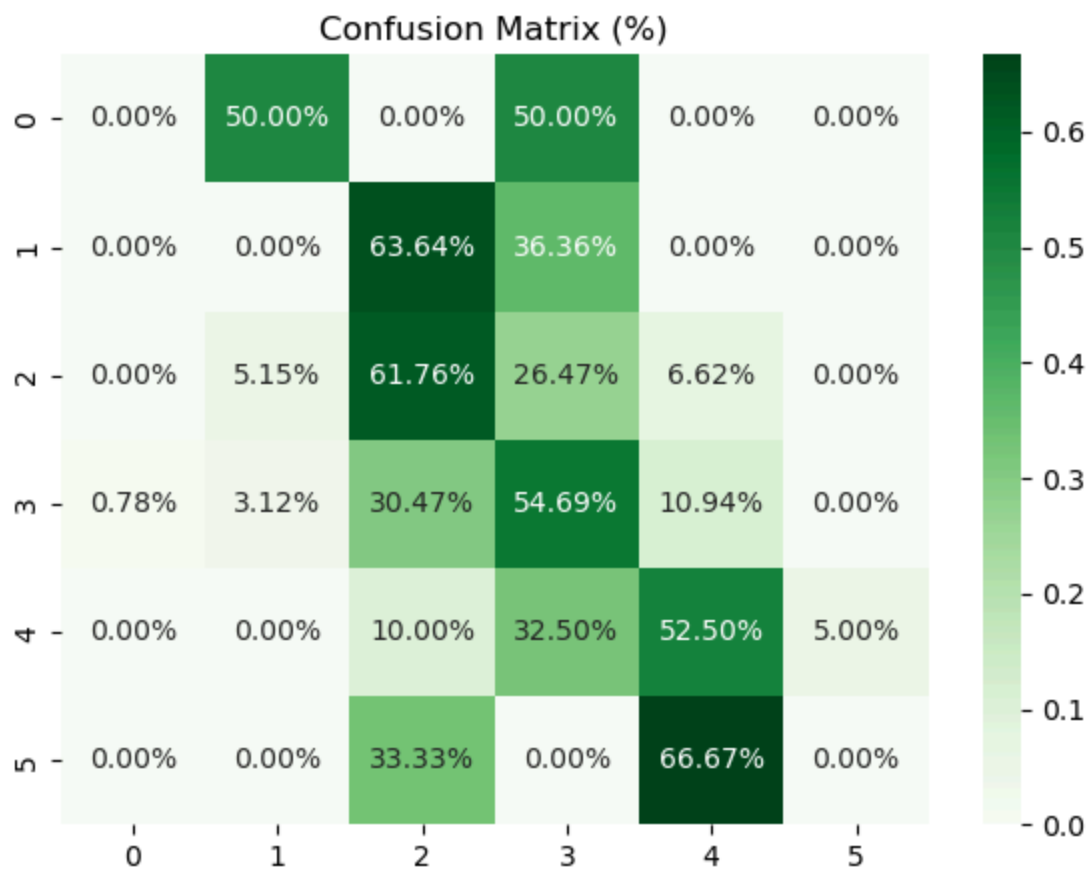
Validation Score

```
In [59]: from sklearn.model_selection import validation_curve
param_range = [2,3,4,5,6,7,8,9,10]
train_score,test_score = validation_curve(DecisionTreeClassifier(criterion='entropy')
plt.plot(param_range,train_score.mean(axis=1),label='Training')
plt.plot(param_range,test_score.mean(axis=1),label='Validation')
plt.legend()
plt.xlabel("Max Depth")
plt.ylabel("Accuracy")
plt.show()
```



Normalized Confusion Matrix

```
In [64]: cm = confusion_matrix(y_test,y_pred)
cm_percent = cm/cm.sum(axis=1,keepdims=True)
plt.figure(figsize=(7,5))
sns.heatmap(cm_percent,annot=True,fmt=".2%",cmap="Greens")
plt.title("Confusion Matrix (%)")
plt.show()
```



In []: